

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Ứng dụng Mô hình ngôn ngữ lớn và Tác tử AI trong
xây dựng hệ thống hỗ trợ học tập

NGUYỄN KHẮC ĐIỆP
diep.nk225806@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: TS. Ngô Văn Linh

Ngành: Công nghệ thông tin

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Ứng dụng Mô hình ngôn ngữ lớn và Tác tử AI trong
xây dựng hệ thống hỗ trợ học tập

NGUYỄN KHẮC ĐIỆP
diep.nk225806@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: TS. Ngô Văn Linh _____

Chữ kí GVHD

Khoa: Công nghệ thông tin Việt-Nhật

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2022

LỜI CẢM ƠN

Để hoàn thành đồ án này, em xin gửi lời cảm ơn chân thành đến quý thầy cô đã tận tâm hướng dẫn và trang bị kiến thức chuyên môn quý báu. Con xin cảm ơn gia đình vì luôn là chỗ dựa tinh thần vô giá. Cảm ơn những người bạn đồng hành và đặc biệt là người yêu đã luôn thấu hiểu, san sẻ áp lực và động viên em trong suốt thời gian qua. Và cuối cùng, tôi muốn gửi lời cảm ơn đến chính bản thân mình vì đã không bỏ cuộc, luôn chăm chỉ, kiên định và quyết tâm nỗ lực vượt qua mọi thử thách để hoàn thiện công việc với kết quả tốt nhất. Sự đồng hành của tất cả mọi người là động lực to lớn giúp em đạt được cột mốc này. Xin trân trọng cảm ơn!

TÓM TẮT NỘI DUNG ĐỒ ÁN

Trong bối cảnh chuyển đổi số giáo dục, học sinh và giáo viên Trung học Phổ thông (đặc biệt môn Tin học) đối mặt với nhiều thách thức. Học sinh cần môi trường học tương tác, còn giáo viên mất nhiều thời gian soạn giáo án, sinh bài tập và chấm điểm. Mặc dù các mô hình ngôn ngữ lớn (LLM) mang lại tiềm năng lớn, chúng vẫn gặp rào cản về việc sinh thông tin sai lệch (hallucination) và không bám sát chương trình chuẩn. Từ đó, bài toán đặt ra là xây dựng một trợ lý ảo giáo dục thông minh có khả năng xử lý tự động, chính xác các tác vụ dựa trên nguồn tri thức chuẩn mực.

Nghiên cứu ứng dụng kiến trúc tìm kiếm và sinh văn bản (RAG) kết hợp phương pháp điều phối đa tác nhân (Multi-Agent LLM). Cụ thể, cơ sở dữ liệu vector được xây dựng từ sách giáo khoa chuẩn. Quá trình truy xuất dữ liệu áp dụng tìm kiếm lai (từ khóa và ngữ nghĩa), tối ưu bằng thuật toán Reciprocal Rank Fusion và mạng Cross-Encoder để xếp hạng lại nhằm trích xuất chính xác ngữ cảnh. Đồng thời, hệ thống bóc tách, đánh giá và định tuyến đa ý định thông qua bộ điều khiển luồng trung tâm (Pipeline Controller). Các dịch vụ miền sau đó xử lý lệnh song song cho các tác vụ như hỏi đáp, sinh bài tập đa định dạng, chấm điểm tự luận theo tiêu chí và tạo cấu trúc bài giảng.

Thử nghiệm trên 246 mẫu kiểm thử bằng khung đo lường RAGAS cho kết quả rất khả quan. Hệ thống đạt tính trung thực (Faithfulness) 0.975 giúp khắc phục gần như hoàn toàn hiện tượng ảo giác, độ phủ ngữ cảnh (Context Recall) đạt 0.959 và mức độ liên quan (Answer Relevancy) đạt 0.857. Tốc độ phản hồi trung bình của toàn hệ thống duy trì ổn định ở mức 6,5 giây cho mỗi truy vấn phức tạp.

Nghiên cứu cung cấp một giải pháp công nghệ giáo dục (EdTech) hiệu quả, hỗ trợ tự động hóa một số quy trình dạy và học môn Tin học THPT. Đo thời, kết quả chứng minh sự thành công của kiến trúc lai RAG và Multi-Agent trong việc định hướng LLM, tạo nền tảng vững chắc để phát triển các hệ sinh thái học tập cá nhân hóa sâu rộng hơn.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

ABSTRACT

In the digital transformation of education, high school students and teachers face notable challenges in Computer Science. Students need interactive self-study environments, while teachers spend excessive time planning lessons, creating quizzes, and grading. Although Large Language Models (LLMs) offer high potential, they suffer critically from information hallucination and a lack of grounding to authorized curricula. Thus, this project proposes an automated, intelligent virtual educational assistant strictly bounded by standardized knowledge bases.

To resolve these challenges, the research introduces a system structured on a Retrieval-Augmented Generation (RAG) architecture orchestrated by a Multi-Agent LLM approach. A vector database was built indexing textbook content from grades 10 to 12. The retrieval pipeline employs a Hybrid Search strategy optimized by Reciprocal Rank Fusion and a Cross-Encoder model for precise Vietnamese document reranking. Driven by a Thin Pipeline Controller that supports dynamic multi-intent routing, domain services parallelly handle complex workflows. These include standard knowledge queries, context-aware quiz generation in varying formats, criteria-based semantic answer grading, and structured lesson plan generation.

Evaluated using the RAGAS framework across 246 comprehensive test instances, the system attained outstanding metrics. It recorded a Faithfulness score of 0.975 (virtually eliminating hallucinations), a Context Recall score of 0.959, and an Answer Relevancy score of 0.857. Furthermore, the framework demonstrated robust performance, achieving an average response latency of 6.5 seconds per multi-intent query.

Ultimately, this thesis presents a scalable EdTech solution that pragmatically automates pedagogical steps for High School Computer Science. It empirically affirms the capability of hybrid retrieval architectures coupled with multi-agent orchestration to mitigate language model hallucinations structurally. This establishes a firm foundation for deploying proactive, personalized learning ecosystems in Vietnam.

Danh mục ký hiệu và chữ viết tắt

Ký hiệu	Ý nghĩa tiếng Anh	Ý nghĩa tiếng Việt
AI	Artificial Intelligence	Trí tuệ nhân tạo
API	Application Programming Interface	Giao diện lập trình ứng dụng
BM25	Best Matching 25	Thuật toán đối sánh tốt nhất 25
BPE	Byte-Pair Encoding	Một thuật toán mã hóa từ vựng
CD		Cánh điều (Sách giáo khoa)
CoT	Chain-of-Thought	Chuỗi tư duy
HyDE	Hypothetical Document Embeddings	Biểu diễn tài liệu giả định
IDF	Inverse Document Frequency	Tần suất nghịch đảo tài liệu
JSON	JavaScript Object Notation	Định dạng dữ liệu văn bản
KNTT		Kết nối tri thức (Sách giáo khoa)
LLM	Large Language Model	Mô hình ngôn ngữ lớn
NLP	Natural language processing	Xử lý ngôn ngữ tự nhiên
Q&A	Question and Answer	Câu hỏi và câu trả lời
RAG	Retrieval-Augmented Generation	Tạo sinh tăng cường bằng truy xuất
RRF	Reciprocal Rank Fusion	Kết hợp thứ hạng nghịch đảo
SGK		Sách giáo khoa
TF	Term Frequency	Tần suất xuất hiện của từ

Danh mục thuật ngữ chuyên ngành

Thuật ngữ	Giải thích ý nghĩa
Agent	<i>(Tác tử)</i> - Hệ thống tự trị có lỗi là trí tuệ nhân tạo, có khả năng diễn dịch yêu cầu, lập suy luận và sử dụng công cụ để thực thi hành động.
Chunk	<i>(Phân đoạn)</i> - Một đoạn văn bản có ý nghĩa hoàn chỉnh được cắt nhỏ từ tài liệu gốc, có kích thước vừa vặn với bộ nhớ của mô hình ngôn ngữ.
Chunking	<i>(Phân mảnh dữ liệu)</i> - Kỹ thuật chia nhỏ các tài liệu quy mô lớn thành các chunk nhằm thuận tiện cho việc lập chỉ mục và tìm kiếm thông tin.
Context Window	<i>(Cửa sổ ngữ cảnh)</i> - Giới hạn tối đa số lượng từ (hoặc token) mà một LLM có thể tiếp nhận và ghi nhớ trong một phiên xử lý duy nhất.
Embedding	<i>(Biểu diễn Vector/Nhúng)</i> - Kỹ thuật mã hóa ngôn ngữ tự nhiên thành các vector số học có độ phân giải cao, giúp máy tính có thể tính toán đo lường mức độ tương đồng ý nghĩa.
Hallucination	<i>(Hiện tượng Ảo giác)</i> - Trạng thái mà mô hình ngôn ngữ lớn đưa ra những câu trả lời có hệ thống về ngoài rất tự tin nhưng hoàn toàn sai sự thật hoặc thiếu căn cứ.
Metadata	<i>(Siêu dữ liệu)</i> - Các thẻ thông tin đi kèm (như Tên bài học, Chương, Chủ đề) nhằm bổ trợ và cung cấp căn cứ nguồn gốc cho các chunk dữ liệu.
Multi-Agent	<i>(Đa tác tử)</i> - Một hệ thống phức hợp nơi nhiều tác tử (agent) hoạt động song song hoặc phối hợp nhằm giải tỏa áp lực của một tác vụ quá lớn.
Prompt	<i>(Chỉ thị)</i> - Câu lệnh hoặc ngữ cảnh đầu vào hướng dẫn mô hình ngôn ngữ lớn thực hiện trả lời, thiết lập phong cách hay xuất ra bằng định dạng nhất định.
Reranker	<i>(Bộ xếp hạng lại)</i> - Mô hình máy học nâng cao làm nhiệm vụ rà soát, tinh chỉnh thứ tự các kết quả tìm ráp ban đầu để đẩy văn bản có liên đới chặt chẽ nhất lên cực đỉnh.

Retriever	<i>(Bộ truy xuất)</i> - Thành phần thuật toán thực hiện tìm kiếm, quét qua toàn bộ cơ sở dữ liệu Vector để lấy ra cụm tài liệu thô gần khớp với truy vấn người dùng.
Token	<i>(Đơn vị cơ sở)</i> - Thành phần nhỏ nhất của một đoạn văn bản (có thể là một từ, âm tiết, hoặc cụm ký tự) mà máy tính có thể tách ra để xử lý.
Vector Database	<i>(Cơ sở dữ liệu Vector)</i> - Cơ sở dữ liệu chuyên dụng được thiết kế nhằm lưu giữ và truy vấn tốc độ cao đối với dữ liệu kiểu vector đa chiều.
Underthesea	<i>(Thư viện NLP tiếng Việt)</i> - Một bộ công cụ mã nguồn mở phụ vụ mạnh mẽ cho nhiều bài toán xử lý ngôn ngữ tự nhiên tiếng Việt, nổi bật là tách từ (Word Segmentation).
Word Segmentation	<i>(Tách từ)</i> - Kỹ thuật phân tách chuỗi văn bản thành hệ thống các từ vựng hợp cấu trúc, đặc biệt quan trọng với ngôn ngữ không có định danh ranh giới sẵn bằng khoảng trắng (ví dụ từ ghép tiếng Việt).

MỤC LỤC

Danh mục ký hiệu và chữ viết tắt.....	i
Danh mục thuật ngữ chuyên ngành	ii
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	2
1.3 Định hướng giải pháp.....	3
1.4 Bố cục đồ án	4
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	6
2.1 Khảo sát hiện trạng	6
2.2 Tổng quan chức năng	7
2.2.1 Biểu đồ use case tổng quát	7
2.2.2 Biểu đồ use case phân rã.....	7
2.3 Đặc tả chức năng	10
2.3.1 Đặc tả UC02 - Hỏi đáp kiến thức SGK.....	10
2.3.2 Đặc tả UC03 - Khởi tạo bộ câu hỏi/bài tập	11
2.3.3 Đặc tả UC05 - Sinh slide bài giảng	12
2.3.4 Đặc tả UC06 - Sinh giáo án.....	13
2.3.5 Quy trình nghiệp vụ	13
2.4 Yêu cầu phi chức năng	18
CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	19
3.1 Kiến thức nền tảng về Trí tuệ Nhân tạo và Xử lý ngôn ngữ tự nhiên.....	19
3.1.1 Tokenization (Mã hóa từ vựng).....	19
3.1.2 Word/Document Embeddings (Biểu diễn Vector)	20
3.1.3 Kiến trúc Transformer và Cơ chế Attention.....	20

3.2 Mô hình ngôn ngữ lớn (Large Language Models - LLM).....	21
3.2.1 Tổng quan và Bản chất dự đoán	21
3.2.2 Kỹ thuật Prompt Engineering	22
3.2.3 Khả năng gọi hàm (Function Calling / Tool Use).....	22
3.2.4 Hiện tượng Hallucination (Ảo giác) trong LLM	22
3.3 Retrieval-Augmented Generation (RAG).....	22
3.3.1 Kiến trúc RAG cơ bản	22
3.3.2 Data Chunking (Phân mảnh dữ liệu)	23
3.3.3 Vector Database (Cơ sở dữ liệu Vector).....	23
3.4 Kỹ thuật truy xuất thông tin nâng cao (Advanced Retrieval).....	23
3.4.1 Lexical Search (Tìm kiếm từ khóa)	23
3.4.2 Semantic Search (Tìm kiếm ngữ nghĩa).....	24
3.4.3 Hybrid Search (Tìm kiếm lai) và thuật toán RRF	25
3.4.4 Query Optimization (Tối ưu hóa truy vấn)	25
3.4.5 Reranking (Xếp hạng lại)	25
3.5 Multi-Agent Systems (Hệ thống đa tác tử).....	25
3.5.1 AI Agent (Tác tử AI)	25
3.5.2 Khung lý thuyết ReAct (Reasoning and Acting)	26
3.5.3 Các kiến trúc Đa tác tử (Multi-Agent Architectures).....	26
3.5.4 Cơ chế thực thi song song (Fan-out / Fan-in).....	27
3.6 Framework Đánh giá hệ thống RAG (RAG Evaluation).....	27
3.6.1 Mô hình LLM-as-a-Judge	27
3.6.2 Bộ khung đo lường RAG (RAG Metrics).....	27

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	29
4.1 Môi trường và dữ liệu thực nghiệm.....	29
4.1.1 Môi trường phát triển	29
4.1.2 Dữ liệu thực nghiệm.....	29
4.2 Tổng quan kiến trúc toàn bộ hệ thống	29
4.3 Thiết kế chi tiết các thành phần hệ thống.....	32
4.3.1 Thiết kế Lớp Ngữ cảnh và Ý định (Context & Intent Layer)	32
4.3.2 Thiết kế Lớp Lập kế hoạch (Planning Layer).....	34
4.3.3 Thiết kế Lớp Thực thi (Execution Layer)	35
4.3.4 Thiết kế Lớp Tri thức (Knowledge Layer)	36
4.3.5 Thiết kế Lớp Sinh phản hồi và Hậu kiểm (Generation & Validation).....	38
4.3.6 Thiết kế Streaming Responesen, History Memory và Logging	39
4.4 Đánh giá hiệu năng và Kết quả thực nghiệm.....	40
4.4.1 Phương pháp đánh giá	40
4.4.2 Kết quả thực nghiệm phân hệ RAG	41
4.4.3 Đánh giá độ trễ hệ thống (Latency).....	42
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	43
5.1 Tự xây dựng bộ máy truy xuất kết hợp (BM25 + Semantic + RRF) từ mức cơ sở	43
5.1.1 Dẫn dắt vấn đề.....	43
5.1.2 Giải pháp đề xuất.....	43
5.1.3 Kết quả đạt được	44
5.2 Thiết kế Điều phối viên mức mã nguồn (Code-level Orchestrator).....	44
5.2.1 Dẫn dắt vấn đề.....	44
5.2.2 Giải pháp đề xuất.....	44

5.2.3 Kết quả đạt được	45
5.3 Kiến trúc Adaptive RAG Agent với suy luận chọn lọc Heuristics.....	45
5.3.1 Dẫn dắt vấn đề.....	45
5.3.2 Giải pháp đề xuất.....	45
5.3.3 Kết quả đạt được	46
5.4 Nhận diện Ý định Hai Cấp (2-Level Intent Detection) giải tỏa độ trễ	46
5.4.1 Dẫn dắt vấn đề.....	46
5.4.2 Giải pháp đề xuất.....	46
5.4.3 Kết quả đạt được	46
5.5 Hệ thống Phản biện khép kín (Self-Reflection bằng QuestionValidator).....	47
5.5.1 Dẫn dắt vấn đề.....	47
5.5.2 Giải pháp đề xuất.....	47
5.5.3 Kết quả đạt được	47
5.6 Chiến lược Quản lý Trạng thái Bộ nhớ linh hoạt (Memory State Strategy)..	47
5.6.1 Dẫn dắt vấn đề.....	47
5.6.2 Giải pháp đề xuất.....	47
5.6.3 Kết quả đạt được	48
5.7 Thiết kế Hệ thống Đa tác nhân (Multi-Agent System) cho quá trình Sinh Slide Tự động	48
5.7.1 Tổng quan kiến trúc: Mô hình Supervisor (Supervisor Pattern) và Lựa chọn Sub-Agent.....	48
5.7.2 OutlineAgent: Thiết lập Dàn ý Tổng quát	49
5.7.3 ContentAgent: Phát triển Nội dung Chi tiết (Cơ chế Fan-out)	49
5.7.4 MediaAgent: Tích hợp Dữ liệu Trực quan.....	49
5.7.5 QuizAgent: Sinh bài tập Hệ thống hóa Kiến thức	49
5.7.6 MergeAgent: Tổng hợp và Kiểm duyệt (Bắt buộc).....	49

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	50
6.1 Kết luận.....	50
6.2 Hướng phát triển.....	50
TÀI LIỆU THAM KHẢO.....	53
PHỤ LỤC.....	54
A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP	54
A.1 Ngành học.....	55
A.2 Đánh dấu (bullet) và đánh số (numering)	55
A.3 Cách thêm bảng	56
A.4 Chèn hình ảnh	56
A.5 Tài liệu tham khảo	57
A.6 Cách viết phương trình và công thức toán học.....	57
A.7 Qui cách đóng quyển.....	57
B. ĐẶC TẢ USE CASE.....	59
B.1 Đặc tả use case “Thống kê tình hình mượn sách”	59
B.2 Đặc tả use case “Đăng ký làm thẻ mượn”	59

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ use case tổng quát	7
Hình 2.2	Biểu đồ phân rã Use Case Hỏi đáp kiến thức SGK	8
Hình 2.3	Biểu đồ phân rã Use Case Khởi tạo bộ câu hỏi/bài tập	8
Hình 2.4	Biểu đồ phân rã Use Case UC04	9
Hình 2.5	Biểu đồ phân rã Use Case Sinh slide bài giảng	9
Hình 2.6	Biểu đồ phân rã Use Case Sinh giáo án	10
Hình 2.7	Sơ đồ luồng hoạt động chức năng Hỏi đáp kiến thức SGK . . .	14
Hình 2.8	Sơ đồ luồng hoạt động chức năng Khởi tạo bài tập	15
Hình 2.9	Sơ đồ luồng hoạt động chức năng Sinh slide và giáo án	17
Hình 3.1	Tổng quan các mô hình tổ chức trong hệ thống Đa tác tử (Multi-Agent)	26
Hình 4.1	Sơ đồ tổng quan kiến trúc hệ thống (System Overview Archi- tecture)	31
Hình 4.2	Chi tiết cấu trúc và luồng xử lý tại Lớp Ngữ cảnh và Ý định . .	33
Hình 4.3	Chu trình ra quyết định và duy trì ngữ cảnh tại Lớp Lập kế hoạch (Planning Layer)	34
Hình 4.4	Sơ đồ vòng lặp Đa tác vụ (Multi-Action Loop) tại Lớp Thực thi	35
Hình 4.5	Cấu trúc xử lý và truy xuất thông tin của Knowledge Layer . .	37
Hình 4.6	Giai đoạn phân luồng Tạo sinh và Hậu kiểm (Block 5)	39
Hình 4.7	Khối thao tác Phản hồi và Khắc ghi tiến trình (Block 6)	40
Hình A.1	Internet vạn vật	56
Hình A.2	Quy cách đóng quyển đồ án	58
Hình A.3	Quy cách đóng quyển đồ án	58

DANH MỤC BẢNG BIỂU

Bảng 2.1	Đặc tả UC02 - Hỏi đáp kiến thức SGK	10
Bảng 2.2	Đặc tả UC03 - Khởi tạo bộ câu hỏi/bài tập	11
Bảng 2.3	Đặc tả UC05 - Sinh slide bài giảng	12
Bảng 2.4	Đặc tả UC06 - Sinh giáo án	13
Bảng 4.1	Kết quả đánh giá hệ thống theo framework Ragas	41
Bảng 4.2	Thống kê thời gian trễ của các khối kiến trúc trong hệ thống .	42
Bảng A.1	Table to test captions and labels.	56

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong bối cảnh cuộc cách mạng công nghiệp lần thứ tư đang diễn ra mạnh mẽ trên toàn cầu, chuyển đổi số đã trở thành một trong những ưu tiên hàng đầu của Việt Nam trong chiến lược phát triển quốc gia. Giáo dục được xác định là một trong những lĩnh vực trọng điểm trong quá trình chuyển đổi số, việc ứng dụng công nghệ thông tin và trí tuệ nhân tạo (AI) vào dạy và học không chỉ là xu hướng tất yếu mà còn là yêu cầu cấp thiết để nâng cao chất lượng giáo dục.

Môn Tin học tại trường trung học phổ thông (THPT) đóng vai trò đặc biệt quan trọng trong chương trình giáo dục Việt Nam. Với chương trình mới bao gồm hai bộ sách giáo khoa Cánh Diều và Kết Nối Tri Thức với Cuộc Sống cho các lớp 10, 11 và 12, nội dung bao phủ nhiều chủ đề chuyên môn từ khoa học máy tính, mạng máy tính đến an toàn thông tin. Tuy nhiên, việc giảng dạy và học tập môn học này vẫn đối mặt với nhiều thách thức đáng kể.

Về phía học sinh, các em thường thiếu một công cụ hỗ trợ tự học hiệu quả và bám sát nội dung sách giáo khoa. Các chatbot AI phổ quát hiện nay như ChatGPT, Claude hay Gemini tuy có khả năng trả lời câu hỏi nhưng không được tối ưu cho dữ liệu sách giáo khoa Việt Nam, dẫn đến tình trạng đưa ra thông tin không chính xác (hallucination). Điều này đặc biệt nguy hiểm trong bối cảnh giáo dục khi mà việc tiếp thu kiến thức sai lệch có thể ảnh hưởng nghiêm trọng đến quá trình học tập. Bên cạnh đó, học sinh cũng thiếu nguồn bài tập đa dạng và phản hồi tức thời được thiết kế chuyên biệt theo từng bài học.

Về phía giáo viên, việc biên soạn tài liệu giảng dạy như câu hỏi trắc nghiệm, bài tập hay cấu trúc slide bài giảng đòi hỏi rất nhiều thời gian và công sức. Trong khi đó, hầu hết các nền tảng e-learning hiện có tại Việt Nam chỉ đóng vai trò là kho lưu trữ và phân phối bài giảng số, chưa có khả năng tự động khởi tạo nội dung thông minh dựa trên yêu cầu của người dùng.

Từ những phân tích trên, nhu cầu xây dựng một hệ thống trợ lý ảo giáo dục ứng dụng công nghệ Tăng cường truy xuất dữ liệu (RAG) sử dụng dữ liệu nền tảng là sách giáo khoa trở nên cấp thiết. Hệ thống này cần đáp ứng đầy đủ các mục tiêu bao gồm tự động trả lời câu hỏi dựa trên nội dung SGK, sinh bài tập đa dạng định dạng, phát thảo cấu trúc slide bài giảng và hỗ trợ giáo viên trong công tác soạn giảng, qua đó tối ưu hóa công việc của cả người dạy lẫn người học.

1.2 Mục tiêu và phạm vi đề tài

Trong những năm gần đây, việc ứng dụng trí tuệ nhân tạo vào giáo dục đã thu hút sự quan tâm của nhiều nhà nghiên cứu và doanh nghiệp trên thế giới. Các mô hình ngôn ngữ lớn (LLM) như ChatGPT, Claude hay Gemini đã chứng minh khả năng ấn tượng trong việc tạo sinh văn bản và trả lời câu hỏi. Tuy nhiên, khi được sử dụng trong bối cảnh giáo dục cụ thể, các mô hình này bộc lộ những hạn chế đáng kể. Đầu tiên, chúng không được huấn luyện trên dữ liệu sách giáo khoa Việt Nam, dẫn đến thường xuyên đưa ra câu trả lời không chính xác hoặc không phù hợp với nội dung chương trình học. Thứ hai, hiện tượng hallucination - tức là sinh ra thông tin nhìn có vẻ đáng tin cậy nhưng hoàn toàn sai sự thật - là vấn đề nan giải mà các LLM hiện tại vẫn chưa thể khắc phục triệt để.

Đối với các nền tảng e-learning trong nước như VnEdu, HOC247, hay Moon, các hệ thống này chủ yếu cung cấp kho tài liệu số và video bài giảng được biên soạn sẵn. Một số nền tảng đã tích hợp tính năng quiz tự động nhưng chưa có khả năng sinh câu hỏi theo yêu cầu cụ thể của người dùng hoặc dựa trên nội dung bài học mới được cập nhật. Việc quản lý và cập nhật ngân hàng câu hỏi vẫn phải thực hiện thủ công, tốn kém nhiều thời gian và công sức.

Về mặt kỹ thuật, công nghệ Retrieval-Augmented Generation (RAG) đã được chứng minh hiệu quả trong việc cải thiện độ chính xác của LLM bằng cách kết hợp khả năng truy xuất tài liệu với khả năng sinh ngôn ngữ tự nhiên. Nhiều nghiên cứu đã áp dụng RAG vào các bài toán giáo dục và đạt được những kết quả khả quan. Tuy nhiên, phần lớn các ứng dụng này tập trung vào các ngữ cảnh sử dụng tiếng Anh và chưa có nhiều nghiên cứu cụ thể cho việc xây dựng hệ thống RAG dựa trên sách giáo khoa Việt Nam. Đặc biệt, việc kết hợp kiến trúc Multi-Agent với RAG để tạo ra hệ thống đa tác vụ (quiz + slide + scoring) vẫn còn là lĩnh vực tương đối mới và chưa được khai thác nhiều trong bối cảnh giáo dục Việt Nam.

So sánh giữa các giải pháp hiện có cho thấy rõ sự khác biệt về năng lực. Trong khi các chatbot AI phổ quát có khả năng đối thoại tự do nhưng không đảm bảo tính chính xác theo SGK, các nền tảng e-learning trong nước có nguồn dữ liệu chuẩn nhưng thiếu khả năng tự động sáng tạo nội dung. Đề tài này hướng tới việc kết hợp ưu điểm của cả hai hướng tiếp cận: đảm bảo tính bám sát sách giáo khoa đồng thời có khả năng sinh tự động các nội dung giáo dục đa dạng.

Dựa trên các phân tích và đánh giá ở trên, đề tài này xác định các hạn chế cần khắc phục bao gồm: thiếu công cụ AI tối ưu cho việc học Tin học THPT tại Việt Nam; chưa có giải pháp RAG nào tích hợp đầy đủ các tác vụ quiz, slide và scoring; và chưa có hệ thống nào kết hợp Multi-Agent với cơ chế Human-In-The-

Loop (HITL) để đảm bảo chất lượng đầu ra trong việc sinh slide bài giảng.

Mục tiêu tổng quát của đề tài là xây dựng hệ thống trợ lý ảo giáo dục hỗ trợ việc dạy và học môn Tin học tại trường trung học phổ thông, sử dụng nền tảng là sách giáo khoa Tin học lớp 10, 11 và 12 thuộc hai bộ sách Cánh Diều và Kết Nối Tri Thức.

Các mục tiêu cụ thể bao gồm: nghiên cứu và xây dựng hệ thống truy xuất kiến thức từ sách giáo khoa dựa trên kiến trúc RAG; thiết kế và phát triển module sinh câu hỏi và bài tập tự động với nhiều định dạng bao gồm trắc nghiệm, tự luận, điền khuyết và đúng/sai; xây dựng module sinh cấu trúc slide bài giảng với kiến trúc đa tác tử; phát triển module chấm điểm và phản hồi tự động dựa trên ngữ cảnh bài học; cuối cùng là đánh giá hiệu năng hệ thống thông qua các chỉ số đánh giá chuẩn.

1.3 Định hướng giải pháp

Để giải quyết các vấn đề đã xác định ở phần trên, đề tài này lựa chọn định hướng giải pháp dựa trên sự kết hợp giữa công nghệ Retrieval-Augmented Generation (RAG) và kiến trúc Multi-Agent. Đây là hai công nghệ tiên tiến trong lĩnh vực AI hiện nay, có khả năng bổ trợ lẫn nhau để tạo ra một hệ thống vừa đảm bảo tính chính xác vừa có khả năng xử lý đa tác vụ phức tạp.

Về nền tảng LLM, hệ thống sử dụng mô hình ngôn ngữ lớn của Google với ưu điểm là chi phí thấp, khả năng hỗ trợ tiếng Việt tốt và chế độ sinh output có cấu trúc ổn định. Mô hình này được đánh giá là phù hợp với mục tiêu của đề tài khi cần tối ưu hóa chi phí trong khi vẫn đảm bảo chất lượng đầu ra cần thiết.

Về thành phần truy xuất, thay vì phụ thuộc vào các thư viện đóng gói, đề tài tự xây dựng bộ máy truy xuất kết hợp giữa thuật toán BM25 và tìm kiếm ngữ nghĩa (Semantic Search). Hai luồng kết quả này được hợp nhất thông qua thuật toán Reciprocal Rank Fusion (RRF). Cách tiếp cận này mang lại sự chủ động trong việc tối ưu hóa thuật toán cho văn bản tiếng Việt và tránh phụ thuộc vào các thư viện có thể gây tốn kém tài nguyên không cần thiết.

Về kiến trúc điều phối, hệ thống thiết kế một Orchestrator tại lớp mức lệnh cơ sở thay vì sử dụng các framework có sẵn. Orchestrator này đóng vai trò là bộ xử lý mỏng, điều phối các thành phần xử lý bao gồm ContextAnalyzer, SessionManager, ActionPlanner và RAG Agent. Ưu điểm của cách thiết kế này là toàn bộ quy trình xử lý được tối ưu về tốc độ, đồng thời quá trình theo vết và debug trở nên dễ dàng hơn.

Về kiến trúc Multi-Agent, hệ thống áp dụng mô hình Supervisor với Supervisor Agent đóng vai trò điều phối, tiếp nhận yêu cầu và phân loại đến các sub-agents

chuyên biệt bao gồm: OutlineAgent cho việc tạo dàn ý bài giảng, ContentAgent cho việc sinh nội dung chi tiết, MediaAgent cho việc tìm kiếm hình ảnh minh họa, QuizAgent cho việc sinh câu hỏi luyện tập, và MergeAgent cho việc tổng hợp kết quả. Hệ thống còn tích hợp cơ chế Human-In-The-Loop (HITL) cho phép người dùng duyệt và chỉnh sửa outline trước khi tiến hành sinh nội dung chi tiết.

Về đóng góp chính, đề tài này tự xây dựng bộ máy truy xuất từ mức cơ sở, thiết kế kiến trúc Adaptive RAG với nhiều chiến lược truy xuất khác nhau, xây dựng hệ thống Multi-Agent cho sinh slide bài giảng với các sub-agents chuyên biệt, và tích hợp cơ chế HITL trong quy trình sinh slide.

1.4 Bố cục đồ án

Phần còn lại của báo cáo đồ án tốt nghiệp này được tổ chức như sau.

Chương 2 trình bày về việc khảo sát thực trạng và phân tích yêu cầu của hệ thống trợ lý ảo giáo dục. Trong chương này, em giới thiệu các tác nhân tham gia vào hệ thống, các Use case tổng quan và Use case phân rã, đồng thời đặc tả chi tiết các luồng chức năng chính bao gồm Hỏi đáp kiến thức SGK, Sinh câu hỏi và bài tập, Chấm điểm và phản hồi, và Sinh slide bài giảng.

Chương 3 giới thiệu về các công nghệ và kiến thức nền tảng được áp dụng trong đồ án. Em trình bày các khái niệm về Trí tuệ nhân tạo (AI) và Mô hình ngôn ngữ lớn (LLM), thuật toán Tokenization và Embedding, kiến trúc Transformer và cơ chế Attention. Ngoài ra, chương này cũng phân tích chi tiết công nghệ RAG bao gồm các thuật toán truy xuất, tìm kiếm lai, cùng với kiến trúc Multi-Agent và các phương pháp đánh giá hệ thống.

Chương 4 trình bày về việc thiết kế và cài đặt hệ thống. Em mô tả kiến trúc tổng thể của hệ thống với nhiều lớp xử lý bao gồm Context & Intent Layer, Planning Layer, Execution Layer, Knowledge Layer, Generation Layer và Response & Memory. Chương này trình bày chi tiết từng thành phần từ ContextAnalyzer, QueryRewriter, IntentRouter, SessionManager, ActionPlanner, cho đến RAG Service với nhiều chiến lược truy xuất, Quiz Service và Slide Service với kiến trúc Multi-Agent Supervisor.

Chương 5 trình bày về các giải pháp kỹ thuật đóng góp chính của đồ án. Em giới thiệu chi tiết về việc tự xây dựng bộ máy truy xuất từ mức cơ sở, thiết kế kiến trúc Orchestrator code-level, mô hình Adaptive RAG Agent với suy luận Heuristics, 2-Level Intent Detection, chiến lược quản lý Memory State, và hệ thống Multi-Agent với Supervisor pattern cho sinh slide bài giảng.

Cuối cùng, phần Kết luận tổng kết các kết quả đạt được của đồ án, đánh giá các

chỉ số hiệu năng, đồng thời đề xuất các hạn chế và hướng phát triển trong tương lai.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương này sẽ trình bày các nội dung liên quan đến việc khảo sát, phân tích hiện trạng để từ đó làm rõ các yêu cầu bài toán đối với hệ thống giáo dục AI. Các phân tích được dựa trên bối cảnh thực tế trong giảng dạy và học tập môn Tin học, qua đó đề xuất các chức năng chính của ứng dụng và mô hình hóa chúng thông qua các biểu đồ Use Case nhằm đưa ra một cái nhìn tổng quan và hệ thống.

2.1 Khảo sát hiện trạng

Trong bối cảnh chuyển đổi số giáo dục, việc ứng dụng trí tuệ nhân tạo (AI) mang lại nhiều tiềm năng to lớn, nhưng quá trình dạy và học môn Tin học THPT (các lớp 10, 11, 12 theo chương trình bộ Kết nối tri thức và Cánh diều) vẫn gặp nhiều thách thức.

Về phía học sinh, các em vẫn thiếu một công cụ hỗ trợ tự học bám sát nội dung chương trình trên lớp. Các chatbot AI phổ quát hiện nay (như ChatGPT, Claude) thường cung cấp câu trả lời phân tán, đôi khi sử dụng kiến thức nằm ngoài phạm vi sách giáo khoa Việt Nam, gây khó khăn cho việc ôn tập chuẩn xác. Bên cạnh đó, học sinh cũng thiếu các nguồn bài tập trắc nghiệm đa dạng được thiết kế chuyên biệt và phản hồi tức thời cho từng bài giảng để củng cố kiến thức một cách nhanh chóng.

Về phía giáo viên, công tác chuẩn bị bài giảng (soạn giáo án, thiết kế slide) và biên soạn đề kiểm tra đang tiêu tốn rất nhiều thời gian, công sức. Các thao tác tìm kiếm, chất lọc tư liệu từ các nguồn rời rạc trên mạng hiện nay vẫn chủ yếu được thực hiện thủ công, lặp đi lặp lại do thiếu vắng các công cụ hỗ trợ tự động hóa chuyên biệt ở mức độ sâu.

Khảo sát các giải pháp e-learning hiện có, phần lớn hệ thống mới chỉ đóng vai trò là kho lưu trữ và phân phối bài giảng số chứ chưa thể chủ động khởi tạo nội dung thông minh. Việc dùng trực tiếp các mô hình ngôn ngữ lớn khổng lồ mà không qua tinh chỉnh lại không đảm bảo được tính chính xác theo khuôn khổ tài liệu chuyên biệt quy định. Vì vậy, nhu cầu cấp thiết hiện nay là xây dựng một hệ thống trợ lý ảo giáo dục ứng dụng công nghệ tăng cường truy xuất dữ liệu (RAG) sử dụng dữ liệu nền tảng là sách giáo khoa. Hệ thống này sẽ đáp ứng trọn vẹn mục tiêu tự động trả lời câu hỏi, sinh bài tập, phát thảo giáo án và cấu trúc slide bài giảng, qua đó tối ưu hóa công việc của cả người dạy lẫn người học.

2.2 Tổng quan chức năng

Dựa trên các phân tích và các kết quả thu được từ tiến trình khảo sát hiện trạng, phần này sẽ mô tả tổng hợp lại các chức năng chính của phần mềm tương ứng với các phân quyền truy cập của người dùng. Hệ thống được thiết kế xoay quanh việc phục vụ nhóm người dùng cốt lõi nhằm giải quyết ba vấn đề lớn đã được nêu rõ thông qua các chức năng liên quan đến mục hỏi đáp kiến thức bài học, sinh biểu mẫu câu hỏi bài tập, tạo khung giáo án giáo khoa và thiết kế bài giảng trình chiếu tự động. Các chức năng chi tiết này sẽ được thiết kế tập trung để quy trình hóa và thiết lập toàn bộ các kết nối một cách hiệu quả nhất có thể. Dưới đây là các biểu đồ thiết kế để diễn giải một cách chi tiết về chức năng cũng như quy trình.

2.2.1 Biểu đồ use case tổng quát

Biểu đồ use case tổng quát mô tả các tác nhân và các chức năng chính của hệ thống.

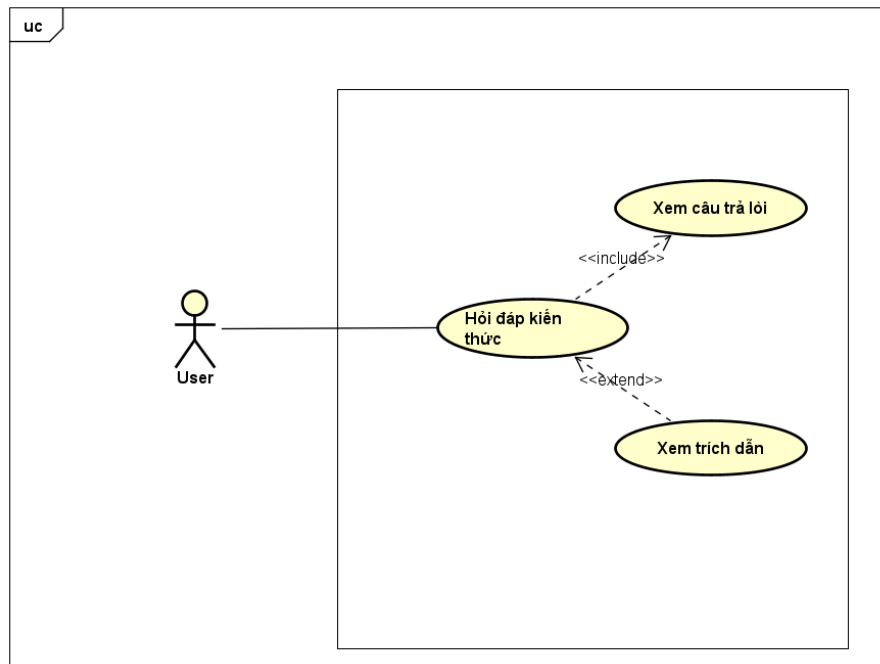


Hình 2.1: Biểu đồ use case tổng quát

2.2.2 Biểu đồ use case phân rã

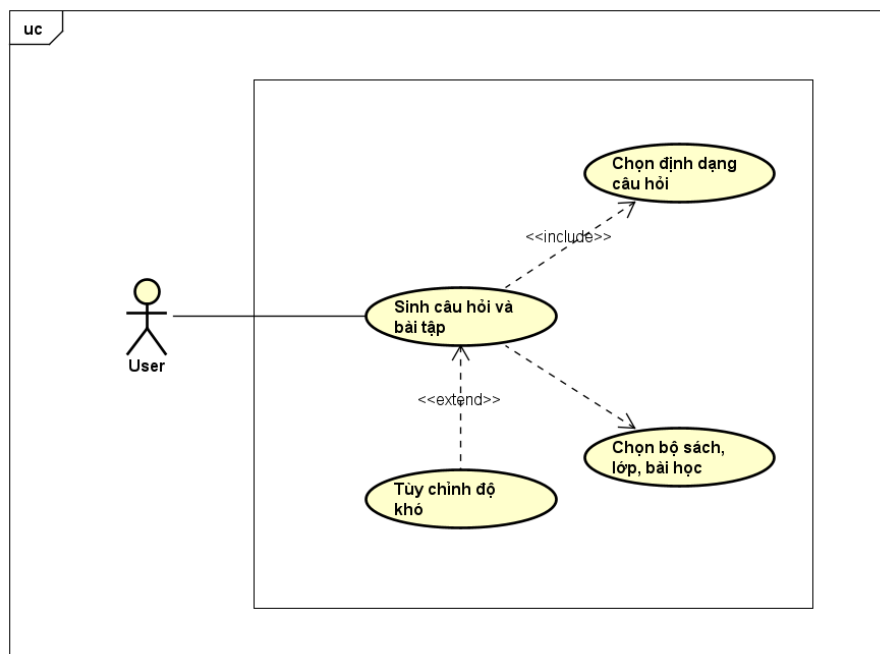
Dưới đây là các biểu đồ phân rã cho các chức năng chi tiết trong hệ thống giáo dục AI.

a, Phân rã UC02 - Hỏi đáp kiến thức SGK



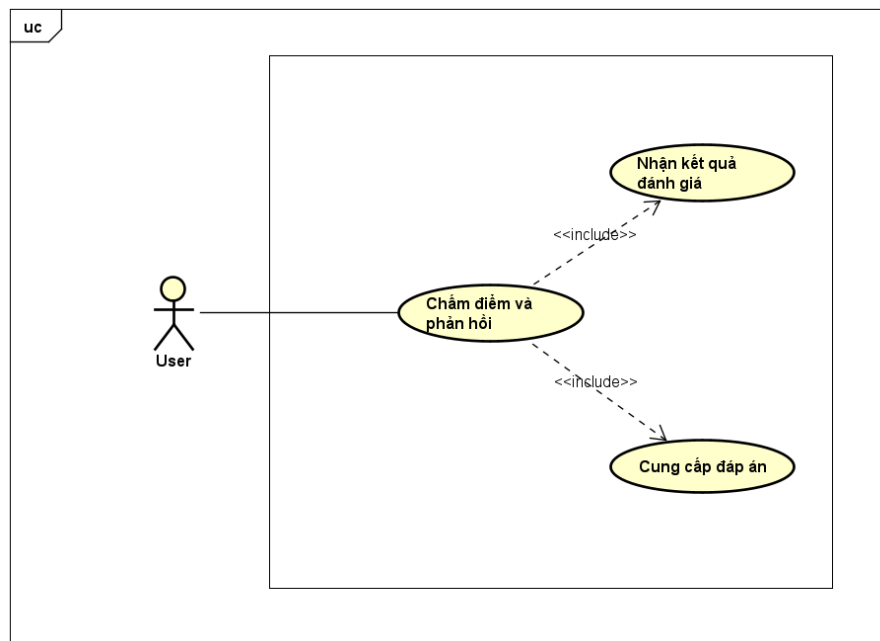
Hình 2.2: Biểu đồ phân rã Use Case Hỏi đáp kiến thức SGK

b, Phân rã UC03 - Khởi tạo bộ câu hỏi/bài tập



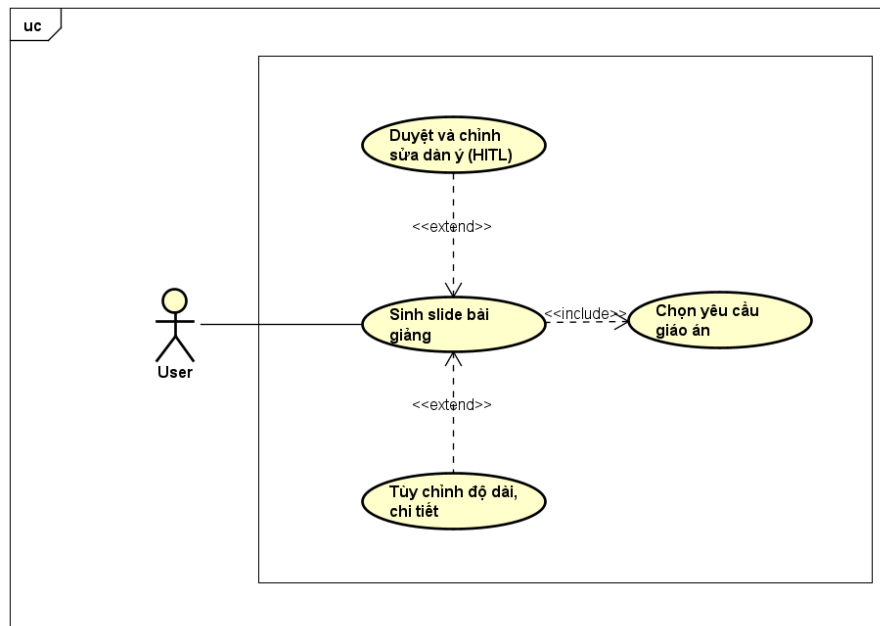
Hình 2.3: Biểu đồ phân rã Use Case Khởi tạo bộ câu hỏi/bài tập

c, Phân rã UC04



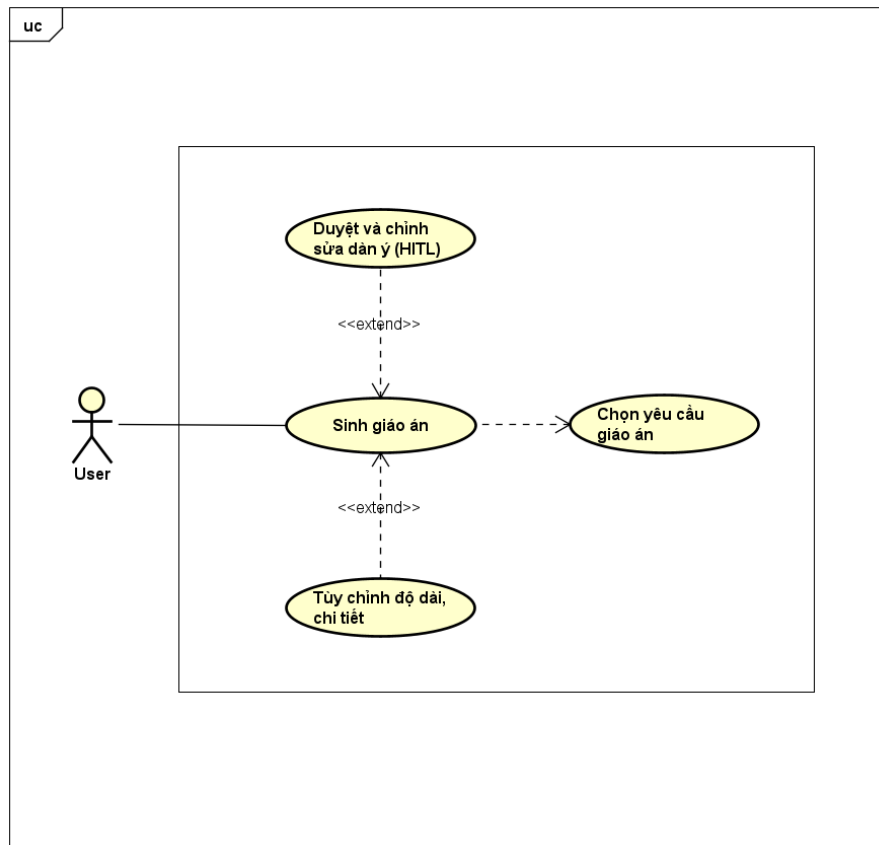
Hình 2.4: Biểu đồ phân rã Use Case UC04

d, Phân rã UC05 - Sinh slide bài giảng



Hình 2.5: Biểu đồ phân rã Use Case Sinh slide bài giảng

e, Phân rã UC06 - Sinh giáo án



Hình 2.6: Biểu đồ phân rã Use Case Sinh giáo án

2.3 Đặc tả chức năng

Dưới đây là đặc tả chi tiết cho các use case quan trọng của hệ thống:

2.3.1 Đặc tả UC02 - Hỏi đáp kiến thức SGK

Bảng 2.1: Đặc tả UC02 - Hỏi đáp kiến thức SGK

Mục	Đặc tả
Tên use case	UC02 - Hỏi đáp kiến thức SGK
Tác nhân	User
Tiền điều kiện	(1) Hệ thống hoạt động bình thường; (2) Kho tri thức SGK đã được nạp; (3) User đang ở giao diện chat

Mục	Đặc tả
Luồng sự kiện chính	1) User nhập câu hỏi kiến thức. 2) Hệ thống phân tích ngữ cảnh hội thoại và nhận diện intent. 3) Hệ thống truy xuất tri thức (RAG) theo chủ đề/lớp/bộ sách. 4) Hệ thống sinh câu trả lời dựa trên ngữ cảnh truy xuất. 5) Hiển thị câu trả lời cho User.
Luồng sự kiện phát sinh	A1: Câu hỏi mơ hồ/phụ thuộc ngữ cảnh → hệ thống sinh đa truy vấn để tăng chất lượng truy xuất. A2: User chưa chọn bộ sách → hệ thống yêu cầu bổ sung thông tin bộ sách trước khi trả lời. A3: Không tìm thấy ngữ cảnh phù hợp → hệ thống trả thông báo và gợi ý cách hỏi lại.
Hậu điều kiện	(1) Câu trả lời được trả về cho User; (2) Lịch sử hội thoại được cập nhật vào session; (3) Thông tin debug/trace được lưu

2.3.2 Đặc tả UC03 - Khởi tạo bộ câu hỏi/bài tập

Bảng 2.2: Đặc tả UC03 - Khởi tạo bộ câu hỏi/bài tập

Mục	Đặc tả
Tên use case	UC03 - Khởi tạo bộ câu hỏi/bài tập
Tác nhân	User
Tiền điều kiện	(1) Hệ thống sẵn sàng; (2) Kho SGK khả dụng; (3) User cung cấp yêu cầu sinh câu hỏi
Luồng sự kiện chính	1) User nhập yêu cầu tạo câu hỏi (chủ đề, dạng câu hỏi, số lượng). 2) Hệ thống nhận diện intent sinh câu hỏi. 3) Hệ thống truy xuất tri thức liên quan từ SGK. 4) Hệ thống sinh bộ câu hỏi theo định dạng yêu cầu. 5) Hiển thị danh sách câu hỏi cho User.

Mục	Đặc tả
Luồng sự kiện phát sinh	A1: User không nêu rõ loại câu hỏi → hệ thống dùng mặc định (ví dụ MCQ). A2: User yêu cầu mức độ khó (planned) → hệ thống điều chỉnh prompt sinh câu hỏi theo mức độ. A3: User muốn chỉnh sửa/xuất file → hệ thống mở rộng thao tác sau khi sinh.
Hậu điều kiện	(1) Bộ câu hỏi được tạo thành công hoặc trả lỗi có hướng dẫn; (2) Trạng thái quiz được lưu vào session để phục vụ chấm điểm/ôn tập tiếp theo

2.3.3 Đặc tả UC05 - Sinh slide bài giảng

Bảng 2.3: Đặc tả UC05 - Sinh slide bài giảng

Mục	Đặc tả
Tên use case	UC05 - Sinh slide bài giảng
Tác nhân	User
Tiền điều kiện	(1) Hệ thống AI và RAG hoạt động; (2) User cung cấp chủ đề bài giảng; (3) Có thông tin lớp/bộ sách (hoặc xác định được tự động)
Luồng sự kiện chính	1) User nhập yêu cầu sinh slide. 2) Hệ thống nhận diện intent tạo slide. 3) Hệ thống truy xuất tri thức SGK liên quan. 4) Supervisor gọi agent tạo outline. 5) Hệ thống sinh nội dung chi tiết từng slide. 6) (Tùy chọn) sinh media và câu hỏi luyện tập. 7) Hệ thống merge kết quả và kiểm tra chất lượng. 8) Trả bộ slide hoàn chỉnh cho User.
Luồng sự kiện phát sinh	A1: User duyệt/chỉnh sửa outline (HITL) trước khi tiếp tục sinh nội dung. A2: Một agent phụ (media/quiz) lỗi → hệ thống tiếp tục với output một phần (partial) nếu lỗi vẫn đạt. A3: Chưa có bộ sách → hệ thống yêu cầu User bổ sung.

Mục	Đặc tả
Hậu điều kiện	(1) Slide được tạo và hiển thị; (2) Dữ liệu slide lưu vào session; (3) Có thể tiếp tục chỉnh sửa/resume trong phiên hiện tại

2.3.4 Đặc tả UC06 - Sinh giáo án

Bảng 2.4: Đặc tả UC06 - Sinh giáo án

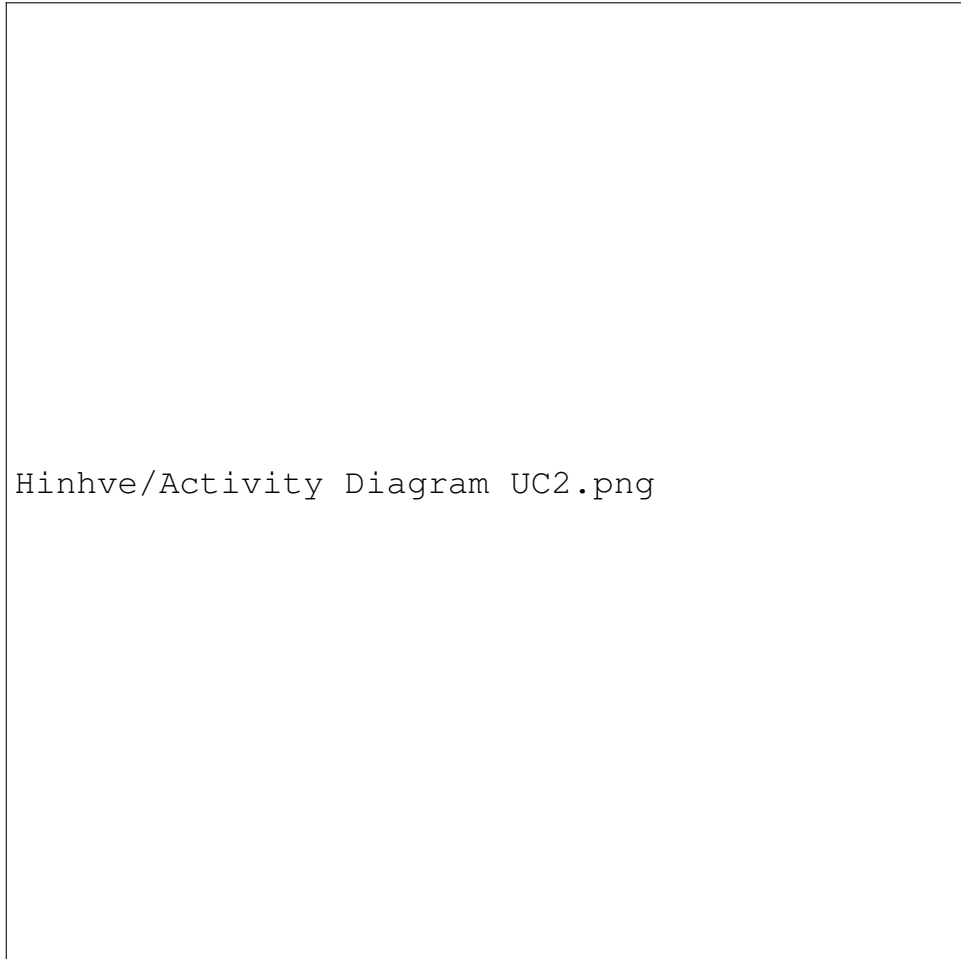
Mục	Đặc tả
Tên use case	UC06 - Sinh giáo án
Tác nhân	User
Tiền điều kiện	(1) Hệ thống hoạt động ổn định; (2) User cung cấp yêu cầu giáo án (chủ đề/lớp/mục tiêu)
Luồng sự kiện chính	1) User nhập yêu cầu sinh giáo án. 2) Hệ thống nhận diện intent sinh giáo án. 3) Hệ thống truy xuất tri thức từ SGK. 4) Hệ thống tạo cấu trúc giáo án. 5) Hệ thống sinh nội dung chi tiết theo cấu trúc. 6) Trả giáo án cho User.
Luồng sự kiện phát sinh	A1: User yêu cầu tùy chỉnh độ dài/mức chi tiết → hệ thống tái sinh nội dung theo tham số mới. A2: User yêu cầu xuất định dạng Markdown/Text → hệ thống thực hiện xuất file. A3: Ngữ cảnh truy xuất chưa đủ → hệ thống yêu cầu làm rõ chủ đề.
Hậu điều kiện	(1) Giáo án được sinh và trả về; (2) Nội dung được lưu vào session; (3) Sẵn sàng cho chỉnh sửa/xuất bản

2.3.5 Quy trình nghiệp vụ

Phần này mô tả các luồng hoạt động (Activity) cốt lõi của hệ thống thông qua các bước tương tác giữa Người dùng (User), Hệ thống điều phối (System) và các Mô hình ngôn ngữ lớn (LLM). Các quy trình được thiết kế nhằm tối ưu hóa khả năng phản hồi tự nhiên của AI dựa trên kiến thức sách giáo khoa có sẵn.

a, Quy trình Hỏi đáp kiến thức SGK

Quy trình này diễn tả cách hệ thống xử lý khi học sinh hoặc giáo viên đặt các câu hỏi truy vấn về kiến thức môn học. Đây là quy trình áp dụng trực tiếp kiến trúc RAG (Retrieval-Augmented Generation).



Hình 2.7: Sơ đồ luồng hoạt động chức năng Hỏi đáp kiến thức SGK

Diễn giải quy trình:

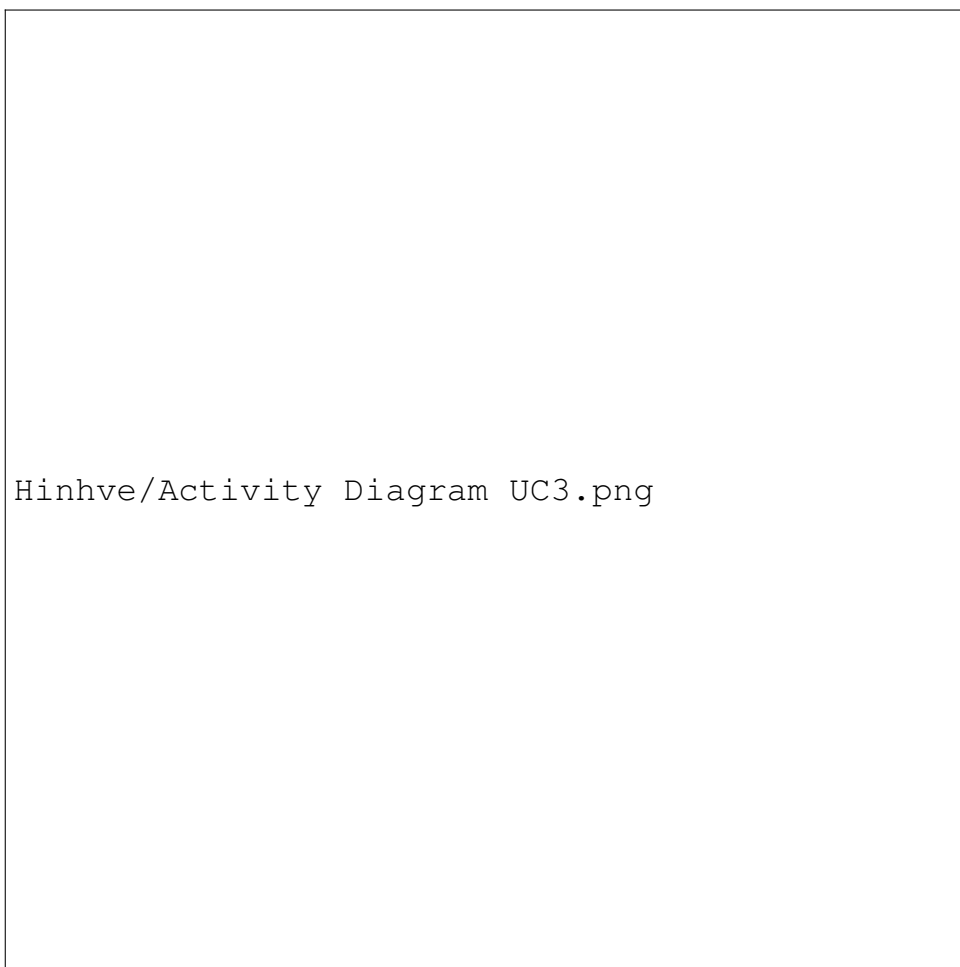
- **Bước 1 (Tiếp nhận):** Người dùng nhập truy vấn vào giao diện chat.
- **Bước 2 (Đánh giá ngữ cảnh):** Hệ thống lấy câu hỏi hiện tại kết hợp cùng lịch sử hội thoại gần nhất đưa vào module Router (LLM) để nhận diện ý định. Nếu ý định được xác định là hỏi đáp, luồng RAG sẽ được kích hoạt.
- **Bước 3 (Viết lại truy vấn):** LLM thực hiện thiết kế lại câu hỏi ban đầu (Query Rewriting) hoặc sinh đa truy vấn (Multi-Query) nhằm làm rõ ngữ nghĩa và loại bỏ sự mơ hồ.
- **Bước 4 (Truy xuất tri thức):** Các câu hỏi sau khi viết lại được chuyển thành các vector đặc trưng (Embedding) và đối chiếu với Vector Database.

Hệ thống áp dụng Hybrid Search (Semantic + BM25) để lấy ra các đoạn tài liệu (chunks) SGK liên quan nhất.

- **Bước 5 (Tổng hợp và xếp hạng):** Tài liệu thô được đưa qua mô hình Reranker để đánh giá chéo độ tương đồng và chọn ra Top K đoạn văn bản chính xác nhất làm ngữ cảnh (Context).
- **Bước 6 (Sinh văn bản):** Hệ thống xây dựng một prompt chứa câu hỏi gốc và ngữ cảnh Top K, sau đó gửi cho LLM để tạo ra câu trả lời cuối cùng. Câu trả lời gửi về cho người dùng và vòng hội thoại mới bắt đầu.

b, Quy trình Khởi tạo bộ câu hỏi/bài tập

Giáo viên hoặc học sinh có thể yêu cầu hệ thống tạo ra một bài kiểm tra hoặc danh sách câu hỏi ôn tập dựa trên một chủ đề cụ thể.



Hình 2.8: Sơ đồ luồng hoạt động chức năng Khởi tạo bài tập

Diễn giải quy trình:

- **Bước 1 (Tiếp nhận yêu cầu):** Người dùng cung cấp tham số tạo bài tập (ví dụ: "Tạo 5 câu hỏi trắc nghiệm bài Mạng máy tính lớp 10").

- **Bước 2 (Trích xuất tri thức):** Tương tự quy trình Hỏi đáp, hệ thống dùng RAG để lấy chính xác phần nội dung lý thuyết liên quan đến bài "Mạng máy tính" từ SGK để làm cơ sở.
- **Bước 3 (Xây dựng Prompt sinh bài tập):** Hệ thống nạp nội dung tài liệu vào cấu trúc prompt đặc thù (nghiêm ngặt về định dạng trả về - Structured Output), yêu cầu LLM đóng vai trò giáo viên để sinh câu hỏi theo độ khó (Bloom's Taxonomy).
- **Bước 4 (Sinh và kiểm chứng):** LLM sinh bộ câu hỏi định dạng JSON/Markdown. Hệ thống sẽ có bước kiểm tra tự động xem dữ liệu trả về có bị lỗi format hay không, nếu lỗi sẽ yêu cầu LLM tự sửa.
- **Bước 5 (Trình bày & Xuất file):** Hệ thống hiển thị danh sách câu hỏi ra giao diện. Người dùng có thể làm trực tiếp hoặc thao tác tải tệp tin về máy.

c, Quy trình Sinh slide bài giảng và Giáo án (Kiến trúc Multi-Agent)

Đây là quy trình phức tạp nhất nhằm hỗ trợ giáo viên tự động hóa việc chuẩn bị tài liệu giảng dạy. Hệ thống sử dụng cơ chế chia để trị (Fan-out) với mô hình đa tác tử (Multi-Agent).

Hìnhve/Activity Diagram UC5_6.png

Hình 2.9: Sơ đồ luồng hoạt động chức năng Sinh slide và giáo án

Diễn giải quy trình:

- **Bước 1 (Nhập chủ đề):** Giáo viên nhập tên bài học hoặc một chủ đề mong muốn để tạo slide/giáo án.
- **Bước 2 (Lập dàn ý - Outline Generation):** Hệ thống dùng RAG lấy mục lục và nội dung cốt lõi của bài học đó, LLM trung tâm (Supervisor) sẽ tiến hành tạo Dàn ý khái quát (Outline) cho giáo án.
- **Bước 3 (Thực thi song song - Fan-out):** Sơ đồ được chia thành 3 nhánh xử lý độc lập do 3 Sub-Agent đảm nhiệm:
 - *Agent 1 (Mục tiêu & Khởi động):* Sinh phần kiến thức trọng tâm, yêu cầu cần đạt.
 - *Agent 2 (Lý thuyết chính):* Chia nhỏ dàn ý thành các slide/tiểu mục lý thuyết chuyên sâu, có ví dụ minh họa từ SGK.
 - *Agent 3 (Luyện tập & Vận dụng):* Đảm nhận việc sinh bài tập trắc

nghiệm, tình huống thảo luận cuối bài.

- **Bước 4 (Tổng hợp - Fan-in):** Ngay khi 3 Agent hoàn thành, module Supervisor sẽ gom tất cả kết quả lại thành một bản nháp hoàn chỉnh duy nhất.
- **Bước 5 (Trình bày & Chỉnh sửa):** Trả kết quả hiển thị cho giáo viên dưới dạng Markdown. Giáo viên có thể yêu cầu hiệu chỉnh độ dài hoặc bổ sung chi tiết trước khi xuất thành file hoàn chỉnh.

2.4 Yêu cầu phi chức năng

Hệ thống Trợ lý ảo Giáo dục cần tuân thủ các yêu cầu phi chức năng sau để đảm bảo chất lượng và trải nghiệm người dùng:

- **Hiệu năng (Performance):** Hệ thống phản hồi các truy vấn RAG của người dùng trong khoảng thời gian lý tưởng dưới 5 giây. Đối với các tác vụ phức tạp như sinh slide, thời gian chờ tối đa không vượt quá 30 giây.
- **Độ tin cậy (Reliability):** Các kiến thức và nội dung được tạo ra phải bám sát
- **Tính dễ dùng (Usability):** Giao diện người dùng (UI) cần hiển thị trực quan, thân thiện (dựa trên Gradio), giúp giáo viên và học sinh dễ dàng tương tác mà không cần qua đào tạo về công nghệ phức tạp.
- **Tính mở rộng và bảo trì (Scalability & Maintainability):** Hệ thống được thiết kế theo kiến trúc module rời rạc thuần mã nguồn (Code-level Orchestrator), cho phép dễ dàng tích hợp thêm các Agent mới hoặc nâng cấp thư viện mà không làm gián đoạn hệ thống hiện tại.

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương này tập trung trình bày các nền tảng kiến thức cốt lõi về Trí tuệ Nhân tạo (AI) và các kỹ thuật tiên tiến được áp dụng để xây dựng hệ thống Trợ lý ảo giáo dục. Các khái niệm được triển khai từ mức cơ bản đến chuyên sâu, bám sát vào những công nghệ thực tế đang được sử dụng trong dự án.

3.1 Kiến thức nền tảng về Trí tuệ Nhân tạo và Xử lý ngôn ngữ tự nhiên

Khái quát về kiến trúc cốt lõi định hình nên các mô hình ngôn ngữ hiện đại.

3.1.1 Tokenization (Mã hóa từ vựng)

Quá trình chuyển đổi văn bản thô thành các đơn vị cơ sở (token) để máy tính hiểu được. Một trong những thuật toán phổ biến nhất hiện nay là **Byte-Pair Encoding (BPE)**. BPE bắt đầu bằng việc coi mỗi ký tự riêng lẻ là một token, sau đó lặp lại nhiều lần quá trình tìm cặp token xuất hiện cạnh nhau nhiều nhất và gộp chúng thành một token mới. Nguyên lý tần suất của BPE có thể biểu diễn như sau:

$$freq(t_a, t_b) = \sum_{w \in D} C(w) \cdot count((t_a, t_b), w) \quad (3.1)$$

Trong đó:

- D : Tập dữ liệu văn bản đào tạo.
- $C(w)$: Số lần xuất hiện của từ w trong D .
- $count((t_a, t_b), w)$: Số lượng cặp token (t_a, t_b) liên kề nhau bên trong từ w .

Cặp (t_a, t_b) có $freq$ lớn nhất sẽ được ghép lại thành một token mới. Quy trình này giúp cân bằng giữa từ vựng kích thước cố định và khả năng biểu diễn từ chưa biết (Out-Of-Vocabulary).

Vietnamese Tokenization (Tách từ tiếng Việt với Underthesea)

Đối với tiếng Việt, do đặc thù ngôn ngữ đơn lập, các âm tiết đứng rời rạc nhưng thường kết hợp tạo thành từ ghép hoặc từ láy mang ngữ nghĩa chặt chẽ. Việc sử dụng các công cụ Tokenization đa ngữ (như BPE) cắt theo khoảng trắng có thể phá vỡ hoàn toàn cấu trúc từ vựng lõi. Để khắc phục, hệ thống tích hợp thư viện NLP **Underthesea** nhằm thực hiện tách bạch ranh giới từ (Word Segmentation) trước khi đưa vào biểu diễn Vector.

Sự cần thiết của tokenizer đặc thù được thể hiện rõ qua ví dụ kinh điển sau đây với câu: *‘Học sinh học sinh học’*.

- **1. Phân tách đa ngữ thông thường (chia theo âm tiết khoảng trắng):**

→ [‘Học’, ‘sinh’, ‘học’, ‘sinh’, ‘học’]

Cách chia này làm mất hoàn toàn phân định ngữ nghĩa ban đầu của câu, khiến mô hình nhầm lẫn giữa chủ thể và cụm danh từ.

- **2. Tách từ tiếng Việt chuyên dụng với Underthesea:**

→ [‘Học_sinh’, ‘học’, ‘sinh_học’]

Hai từ ghép ‘*học sinh*’ (danh từ chỉ người) và ‘*sinh học*’ (danh từ chỉ môn học) được phát hiện liền mạch và nối nối với nhau chuẩn xác bằng dấu gạch dưới, giúp mô hình đọc hiểu đúng đại ý gốc một cách dễ dàng.

Quá trình tách từ đóng vai trò tiền xử lý cực kỳ quan trọng, cung cấp đầu vào chất lượng cao và bảo toàn trọn vẹn mạch ý nghĩa của văn bản tiếng Việt.

3.1.2 Word/Document Embeddings (Biểu diễn Vector)

Embeddings là cách máy tính biểu diễn token hoặc toàn bộ văn bản dưới dạng vector số học dày đặc (Dense Vector) gồm các số thực liên tục nhằm lưu giữ đặc trưng ngữ nghĩa. Các từ có nghĩa tương đồng sẽ nằm gần nhau hơn trong một không gian vector đa chiều (ví dụ 512, 768 chiều). Để đo mức độ tương đồng giữa hai vector mô tả ngữ nghĩa, ta sử dụng công thức **Cosine Similarity (Độ tương đồng Cosin)**:

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (3.2)$$

Trong đó:

- $\mathbf{A}, \mathbf{B}$: Là hai vector đặc trưng nhiều chiều (n-chiều) biểu diễn cho hai văn bản.
- A_i, B_i : Thành phần thứ i của vector $\mathbf{A}$ và $\mathbf{B}$.
- $\|\mathbf{A}\|, \|\mathbf{B}\|$: Độ dài (norm L2) của mỗi vector.

Giá trị cosin nằm trong khoảng $[-1, 1]$, trong đó 1 chỉ ra hai vector cùng hướng (ý nghĩa giống nhau hoàn toàn), 0 là trực giao (không liên quan), và -1 là ngược hướng.

3.1.3 Kiến trúc Transformer và Cơ chế Attention

Kể từ khi ra mắt vào năm 2017, kiến trúc Transformer đã trở thành sợi chỉ đỏ của NLP hiện đại. Thay vì xử lý tuần tự (như RNN/LSTM), Transformer xử lý văn bản song song thông qua cơ chế Tự chú ý (Self-Attention). Cơ chế **Scaled Dot-Product Attention** cho phép mô hình đánh giá tầm quan trọng của các từ trong câu khi đang

xét một từ bất kỳ. Công thức lõi của cơ chế này là:

$$Attention(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (3.3)$$

Trong đó:

- Q (Query - Truy vấn): Vector biểu diễn từ đang cần tìm độ tương quan phân tích (thứ ta đi tìm).
- K (Key - Khóa): Vector biểu diễn để đối chiếu độ tương hợp với Query cho mọi từ trong câu (thứ ta kiểm tra).
- V (Value - Giá trị): Vector mang nội dung ngữ nghĩa thực sự của từ đó sẽ được giữ lại.
- d_k : Chiều không gian của vector Key, việc chia cho $\sqrt{d_k}$ giúp thu nhỏ phương sai của Tích vô hướng (dot product) tránh việc hàm softmax rơi vào vùng bão hòa gradient (gradient vanishing).
- Hàm softmax đảm bảo ma trận trọng số luôn có phạm vi $[0, 1]$ và tổng phân bố là 1.

3.2 Mô hình ngôn ngữ lớn (Large Language Models - LLM)

Khái quát về thành phần tạo nên "bộ não" suy luận của hệ thống.

3.2.1 Tổng quan và Bản chất dự đoán

Sự tiến hóa từ các mô hình biểu diễn ngữ nghĩa thành các LLM có khả năng suy luận mạnh mẽ là kết quả của việc gia tăng kích thước mạng nơ-ron và dữ liệu sinh văn bản (Causal Language Modeling). Bài toán cốt lõi của LLM là **Dự đoán token tiếp theo (Next-token Prediction)** dựa trên chuỗi từ đã có. Xác suất tạo ra một câu chữ chiều dài T được định nghĩa theo chuỗi phân phối xác suất có điều kiện:

$$P(w_1, w_2, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_1, \dots, w_{t-1}) \quad (3.4)$$

Trong đó:

- w_t : Token mục tiêu tại vị trí t .
- $w_1, \dots, w_{t-1}$: Các token tiền cảnh (context) đã được cung cấp (từ prompt sinh ra phía trước).

Mô hình sẽ chọn token tiếp theo thông qua việc lấy mẫu (Sampling) theo hàm phân phối xác suất này.

3.2.2 Kỹ thuật Prompt Engineering

Là quá trình thiết kế, điều chỉnh các chỉ thị đầu vào (prompt) nhằm định hướng LLM sinh ra câu trả lời theo ý muốn mà không phải đào tạo lại trọng số. Một số kỹ thuật cốt lõi:

- **Zero-shot:** Yêu cầu trả lời trực tiếp mà không cung cấp mẫu trước.
- **Few-shot learning:** Cung cấp một vài ví dụ input-output để mô hình nội suy quy luật.
- **Chain-of-Thought (CoT):** Chỉ thị mô hình suy luận từng bước thay vì nhảy ngay ra kết quả cuối cùng (giúp giảm thiểu sai sót toán học/logic).

3.2.3 Khả năng gọi hàm (Function Calling / Tool Use)

Đây là đặc tính quan trọng giúp LLM kết nối với thế giới bên ngoài thay vì chỉ sinh text bị cô lập. Khi cấu hình một prompt đi kèm danh sách các "Tool", mô hình khi cần dữ liệu hoặc thực hiện thao tác sẽ tự động trả về chuỗi văn bản dạng cấu trúc (JSON) mô tả tên hàm và danh sách tham số (Arguments) thay vì sinh chuỗi text tự do.

3.2.4 Hiện tượng Hallucination (Ảo giác) trong LLM

Bản chất của LLM là dự đoán chuỗi từ mang ý nghĩa tự nhiên nhất, chúng không "nhớ" thông tin theo các bản ghi chính xác. Vì thế, khi đối diện với thông tin chuyên ngành mới hoặc tự suy đoán sự kiện, LLM gặp ảo giác — sinh ra thông tin thoát nhìn rất đáng tin cậy nhưng sai hoàn toàn sự thật. Nguyên nhân cốt lõi do dữ liệu đào tạo (out-of-date) và sự thiếu vắng nền tảng bám trụ kiến thức thực tế (lack of grounding). Điều này là căn cứ mạnh mẽ để đưa công nghệ RAG vào hệ thống.

3.3 Retrieval-Augmented Generation (RAG)

Thành phần quan trọng nhất giúp hệ thống trả lời chính xác dựa trên Sách giáo khoa trực tiếp.

3.3.1 Kiến trúc RAG cơ bản

Nguyên lý hoạt động cơ bản của RAG gồm 3 bước tuần tự: Truy xuất (Retrieval) - Bổ sung (Augmented) - Sinh văn bản (Generation).

- **Pha Offline (Chuẩn bị dữ liệu):** Tập hợp tài nội dung tài liệu, phân mảnh nhỏ (chunking), sau đó quy đổi mảnh bằng Embedding Model và lưu trữ vào Vector Database.
- **Pha Online (Truy vấn):** Nhận câu hỏi thực, mã hóa vector câu hỏi đưa đi tìm kiếm chuỗi nội dung giống nhất (Retrieval), đưa đoạn văn bản kết quả thu được bổ sung vào Prompt hiện tại (Augmented) rồi đưa vào tính toán trả

kết quả bởi LLM (Generation).

3.3.2 Data Chunking (Phân mảnh dữ liệu)

Do các LLM có hạn chế về kích cỡ cửa sổ khung ngữ cảnh (Context Window), nguyên văn tài liệu bắt buộc phải được chia nhỏ (Chunking). Các chiến lược chia nhỏ thường gặp:

- **Fixed-size Chunking:** Cắt nội dung theo chuỗi số lượng token nhất định liên tiếp, có dải đè (overlap) để nối mạch văn.
- **Recursive Character Chunking:** Cắt theo phân cấp ngắt đoạn lô-gic như đoạn văn ($\backslash n \backslash n$), câu xuống dòng ($\backslash n$), khoảng trắng.
- **Hierarchical Chunking (Phân mảnh phân cấp):** Rạch rời dựa trên cấu trúc chương/mục nhằm gắn kết Metadata để duy trì cây tri thức vững chắc mà không làm đứt gãy luồng thông tin văn bản.

3.3.3 Vector Database (Cơ sở dữ liệu Vector)

Nơi lưu trữ Vector Embeddings và hỗ trợ việc tìm kiếm đánh giá mức độ tương đồng giữa không gian các vector. Điểm vượt trội cốt lõi của cơ sở dữ liệu Vector so với các hệ quản trị CSDL quan hệ truyền thống là khả năng tính toán ưu việt chuẩn xác khoảng cách và tọa độ (ví dụ *Euclidean*, *Dot-Product*, *Cosine*) giữa Vector Truy vấn (Query Vector) và hàng triệu Vector gốc của các mảnh tài liệu (Document Vectors), nhằm trả về nhanh chóng các ngữ nghĩa đối sánh nhất.

3.4 Kỹ thuật truy xuất thông tin nâng cao (Advanced Retrieval)

Đi sâu vào các thuật toán tối ưu hóa việc tìm kiếm tài liệu từ kho Sách giáo khoa (Corpus).

3.4.1 Lexical Search (Tìm kiếm từ khóa)

Thuật toán cơ sở dùng để tra cứu sự tương đồng phần chữ cái đối với từ khóa hiếm/chuyên ngành. Nổi bật là cơ chế **TF-IDF** (Term Frequency - Inverse Document Frequency). **Công thức TF-IDF cơ bản:**

$$TF_IDF(t, d, D) = TF(t, d) \times IDF(t, D) \quad (3.5)$$

Trong đó:

- $TF(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$: Tần suất (mật độ) từ t xuất hiện trong văn bản d .
- $IDF(t, D) = \log \frac{N}{|\{d \in D: t \in d\}|}$: Cơ chế đo lường mức độ đặc trưng chuyên biệt với việc nghịch đảo số tài liệu liên quan (N là tổng số văn bản).

Tuy nhiên, TF-IDF không có điểm trần và dễ bị thiên lệch chiều dài văn bản.

Hiện tại phiên bản tiến tiến hơn được sử dụng là **BM25 (Best Matching 25)** với giới hạn tần suất:

$$Score_{BM25}(Q, d) = \sum_{q_i \in Q} IDF(q_i) \cdot \frac{f(q_i, d) \cdot (k_1 + 1)}{f(q_i, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{avgdl}\right)} \quad (3.6)$$

Trong đó:

- Q : Câu truy vấn bao gồm các từ khóa q_i .
- $f(q_i, d)$: Số lần xuất hiện từ khóa q_i trong văn bản d .
- $|d|$ và $avgdl$: Chiều dài độ dài văn bản d và trung bình cộng độ dài hệ thống.
- k_1 và b : Các siêu tham số tinh chỉnh độ dốc phạt do độ dài văn bản và trần bão hòa của tần suất điểm BM25.

3.4.2 Semantic Search (Tìm kiếm ngữ nghĩa)

Thay vì tra xét từ khóa theo dạng văn bản bề mặt, *Semantic Search* đánh giá sự tương đồng trực tiếp về mặt ý nghĩa. Cả truy vấn q và văn bản d trong hệ thống đều được mô hình nhúng ánh xạ thành các dense vector V_q và V_d trong không gian n chiều. Sự tương đồng ngữ nghĩa sau đó được xác định thông qua các độ đo khoảng cách hình học cốt lõi.

Hai biến thể phương trình phổ biến nhất trong hệ thống Vector Search:

1. Tích vô hướng (Dot Product) Đo lường trực tiếp độ lớn và mức độ đồng hướng của hai vector:

$$Score_{dot}(q, d) = V_q \cdot V_d = \sum_{i=1}^n V_{q,i} V_{d,i} \quad (3.7)$$

Phép đo này tính toán cực kỳ nhanh và phù hợp khi các vector đã được chuẩn hóa độ dài (normalized).

2. Độ tương đồng Cosine (Cosine Similarity)

Độ tương đồng Cosine được sử dụng để đo lường góc giữa vector truy vấn và vector tài liệu, qua đó giảm ảnh hưởng của độ dài văn bản và chỉ tập trung vào mức độ tương đồng về mặt ngữ nghĩa giữa các vector.

Điểm số xếp hạng của tài liệu lúc này được xác định trực tiếp thông qua giá trị của hàm Cosine Similarity, như đã được trình bày tại **Công thức 3.2**.

Đoạn tài liệu nào tạo ra độ tương đồng cao nhất với vector truy vấn theo một trong hai thang đo trên sẽ được hệ thống Vector Search ưu tiên trích xuất và chuyển

sang giai đoạn xếp hạng tiếp theo.

3.4.3 Hybrid Search (Tìm kiếm lai) và thuật toán RRF

Mục đích kết hợp sự chính xác ngữ pháp vững từ BM25 và sự nhận thức khái niệm từ Vector Search (Semantic). **Reciprocal Rank Fusion (RRF)** là một thuật toán tích hợp ưu điểm của bảng xếp thứ hạng của hai chiến lược tìm kiếm. Cách tính điểm RRF bù trừ của một đoạn tìm kiếm kết quả d :

$$Score_{RRF}(d) = \frac{1}{k + rank_{BM25}(d)} + \frac{1}{k + rank_{Vector}(d)} \quad (3.8)$$

Trong đó:

- $rank_{BM25}(d)$, $rank_{Vector}(d)$: Trật tự xếp hạng của văn bản d truy được dựa theo từng cơ chế.
- k : Hằng số giảm xóc (thường từ 60).

3.4.4 Query Optimization (Tối ưu hóa truy vấn)

Tiền xử lý dùng nội tính của LLM nâng cao khả năng câu hỏi:

- **Query Rewriting (Viết lại truy vấn)**: Chuyển câu hỏi thô thành các keywords chuyên biệt bám bản thông tin tài liệu.
- **HyDE (Hypothetical Document Embeddings)**: Yêu cầu mô hình tưởng tượng (hallucinate) nội suy nhanh giả câu trả lời, sử dụng đoạn đó làm khung Vector đi tìm phần tham chiếu sát thực tiễn để tăng phần cover (độ phủ tìm kiếm).

3.4.5 Reranking (Xếp hạng lại)

Thay Bi-Encoder bằng việc dùng **Cross-Encoder** đánh bảng điểm Top 10-20 về lại. Bi-Encoder Embedding lưu rõ hai chuỗi làm chậm chạp thiếu nhạy bén trong ngữ cảnh. Khối Cross-Encoder cho trượt luôn cặp chuỗi ký tự Text gốc vào một lớp Self-Attention làm cho sự đánh giá trở nên cực kỳ tinh xác (tất nhiên chi phí truy vấn thời gian nhỉnh hơn bởi đối sánh chéo đôi).

3.5 Multi-Agent Systems (Hệ thống đa tác tử)

Lý thuyết phục vụ cho tính năng thiết kế Bài giảng, Slide hoặc cấu trúc Giáo án tự động.

3.5.1 AI Agent (Tác tử AI)

Triển khai môi trường nơi mà Agent lấy tư duy từ lõi LLM kèm hệ Tools (Công cụ tương tác được ra API hay code bên ngoài – ví dụ đọc code, gọi python tính, trích xuất wiki).

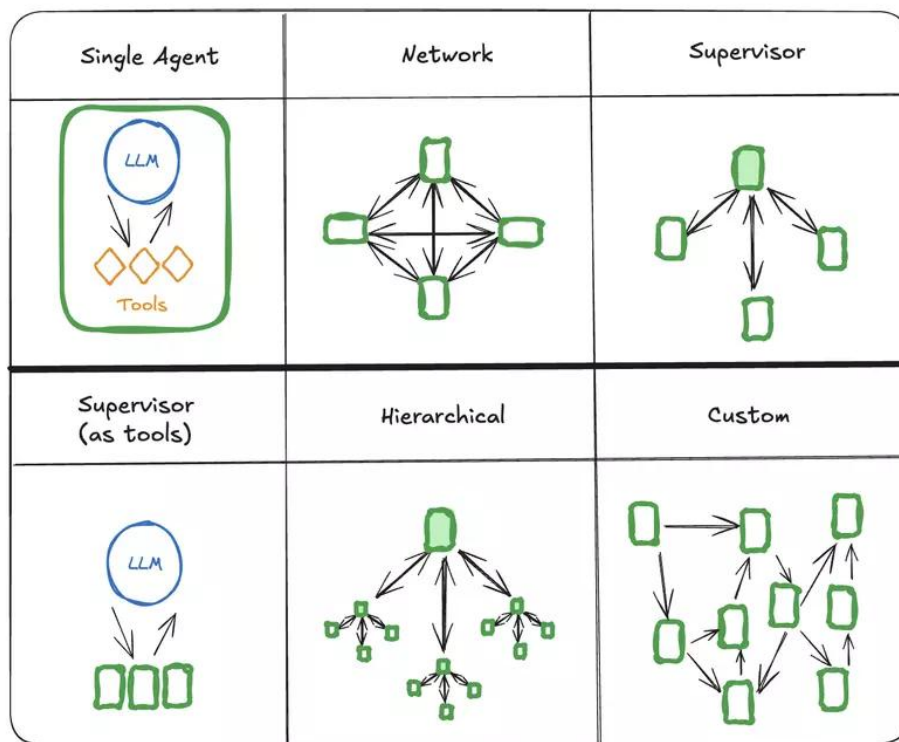
3.5.2 Khung lý thuyết ReAct (Reasoning and Acting)

Một vòng lặp giải bài toán chia tuần tự:

- **Thought (Tư duy):** Nhận xét phương hướng tương ứng tiếp diễn.
- **Action (Hành động):** Điều động chạy tham số gọi Tool cung cấp.
- **Observation (Quan sát):** Kịch bản quan sát phản hồi kết xuất rồi chạy vòng nghĩ Thought để ra trả lời kết.

3.5.3 Các kiến trúc Đa tác tử (Multi-Agent Architectures)

Thay vì phụ thuộc vào một mô hình ngôn ngữ duy nhất để thực hiện toàn bộ tác vụ từ A đến Z, hệ thống đa tác tử (Multi-Agent) chia nhỏ bài toán phức tạp thành nhiều phần và giao cho từng tác tử chuyên biệt xử lý. Việc tổ chức các tác tử này thành một kiến trúc luồng dữ liệu hợp lý là chìa khóa để vận hành hệ thống trơn tru.



Hình 3.1: Tổng quan các mô hình tổ chức trong hệ thống Đa tác tử (Multi-Agent)

Như được trình bày ở Hình 3.1, có rất nhiều kiểu kiến trúc điều phối mạng lưới AI Agent khác nhau, bao gồm:

- **Mô hình mạng lưới (Network):** Các tác tử giao tiếp tự do với nhau không qua trung gian. Cấu trúc này linh hoạt nhưng dễ rối loạn luồng thông tin khi hệ thống phình to.

- **Mô hình hội đồng (Joint / Committee):** Các tác tử cùng ngồi lại báo cáo và thảo luận giải quyết cùng một vấn đề.
- **Mô hình dạng chuỗi (Chain / Sequential):** Output của tác tử này là Input của tác tử kế tiếp, theo một đường truyền cứng.
- **Mô hình Giám sát (Supervisor / Hierarchical):** Một tác tử đóng vai trò nhạc trưởng (Nhà quản lý) đứng ở vị trí trung tâm, chịu trách nhiệm nhận yêu cầu, phân loại và gọi các tác tử chuyên biệt bên dưới thực thi.

Mô hình Supervisor (Điều phối tác tử) và lựa chọn kiến trúc

Trong đồ án này, kiến trúc **Supervisor** được chọn làm thiết kế lõi cho hệ thống trợ lý học tập. Một Supervisor Agent sẽ tiếp nhận câu hỏi, phân tích ý định thay vì xử lý trực tiếp, sau đó định tuyến đến các Agent chuyên sâu tương ứng (VD: Tìm lý thuyết, Sinh bài tập...).

Việc lựa chọn này mang lại 3 lợi thế cốt lõi: (1) **Kiểm soát luồng chặt chẽ**, giúp hệ thống suy luận minh bạch, tránh LLM lạc đề hoặc vòng lặp vô hạn; (2) **Tính đóng gói & phân quyền**, giới hạn quyền truy cập tài nguyên CSDL rõ ràng cho từng Agent; (3) **Dễ dàng mở rộng (Scalability)**, cho phép nhanh chóng thêm các Agent chức năng mới sau này mà không làm xáo trộn kiến trúc tổng thể.

3.5.4 Cơ chế thực thi song song (Fan-out / Fan-in)

Cách chia song song. (Phối cử sub-agent tìm Mở Đầu độc lập với sub-agent tìm Bài Tập) và sau đó tổng kết dồn thông tin (Fan-in) quy tụ làm tiết kiệm mạnh chi phí chờ.

3.6 Framework Đánh giá hệ thống RAG (RAG Evaluation)

Cách đo lường và định lượng bằng số (Metrics) độ chính xác của hệ thống AI bằng các framework chuẩn (ví dụ RAGAS).

3.6.1 Mô hình LLM-as-a-Judge

Sử dụng LLM đóng vai giám khảo cho việc tính hệ số chính xác không can thiệp sức người.

3.6.2 Bộ khung đo lường RAG (RAG Metrics)

Để định lượng hiệu năng của hệ thống RAG, các framework hiện đại (như RAGAS) chia việc đánh giá thành 4 thang đo độc lập, giúp kiểm tra cả chất lượng truy xuất tài liệu lẫn năng lực sinh văn bản của LLM:

- **Độ trung thực (Faithfulness):** Đảm bảo mọi luận điểm (claims) có trong câu trả lời sinh ra đều có thể được chứng minh bởi tài liệu ngữ cảnh (context)

truy xuất được, chống lại hiện tượng ảo giác (hallucination).

$$Faithfulness = \frac{|V \cap C|}{|V|} \quad (3.9)$$

Trong đó, V là tập các luận điểm rút ra từ câu trả lời của LLM, và C là tập các luận điểm có thể suy diễn trực tiếp từ ngữ cảnh.

- **Mức độ bám sát câu hỏi (Answer Relevance):** Đo lường mức độ câu trả lời giải quyết trực tiếp vấn đề được đặt ra trong câu hỏi của người dùng, không thừa thãi hay đi lệch trọng tâm.

$$Answer\ Relevance = \frac{1}{N} \sum_{i=1}^N \cos(E_q, E_{g_i}) \quad (3.10)$$

Trong đó, từ câu trả lời, tiến hành sinh ngược ra N câu hỏi giả định g_i . Sau đó tính trung bình độ tương đồng Cosine giữa vector nhúng của câu hỏi gốc E_q và các câu hỏi giả định E_{g_i} .

- **Độ bao phủ câu trả lời (Context Recall):** Đo lường việc hệ thống tìm kiếm có lấy về đủ mọi thông tin cần thiết để giải quyết câu hỏi mẫu (Ground Truth) hay không.

$$Context\ Recall = \frac{|GT \cap R|}{|GT|} \quad (3.11)$$

Trong đó, GT là tập thông tin thiết yếu có trong đáp án mẫu (Ground Truth), và R là tập thông tin được tìm thấy trong ngữ cảnh được truy xuất (Retrieved Context).

- **Độ chính xác truy xuất (Context Precision):** Đánh giá khả năng xếp hạng (ranking) của hệ thống tìm kiếm, xem những tài liệu chứa thông tin hữu ích nhất có được xếp ở các vị trí đầu tiên hay không.

$$Context\ Precision = \frac{\sum_{k=1}^K (Precision@k \times v_k)}{\text{Tổng số tài liệu liên quan trong } K} \quad (3.12)$$

Trong đó, K là tổng số tài liệu được truy xuất, $v_k = 1$ nếu tài liệu ở vị trí k có chứa đáp án và $v_k = 0$ nếu ngược lại. Precision@k là độ chính xác tính đến vị trí thứ k .

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

4.1 Môi trường, dữ liệu và phạm vi thực nghiệm

Chương này trình bày kết quả hiện thực hệ thống và các thực nghiệm đã thu được trên phiên bản mã nguồn hiện tại. Khác với bản thảo cũ, kiến trúc runtime của hệ thống không còn được mô tả như một pipeline agent chung dựa hoàn toàn trên LangGraph. Luồng xử lý chính hiện được tổ chức theo mô hình *code-level orchestrator*: các bước phân tích ngữ cảnh, quản lý phiên, lập kế hoạch hành động và định tuyến nghiệp vụ được kiểm soát trực tiếp bằng Python; LLM chỉ được dùng ở những điểm cần suy luận ngôn ngữ hoặc sinh nội dung.

4.1.1 Môi trường phát triển

Hệ thống được phát triển bằng Python 3.12. Các thành phần chính của môi trường thực nghiệm gồm:

- **LLM**: Google Gemini thông qua thư viện `google-generativeai`; cấu hình mặc định trong mã nguồn hiện tại là `gemini-2.5-flash-lite`.
- **Embedding**: mô hình `dangvantuan/vietnamese-document-embedding`, sinh vector 768 chiều cho văn bản tiếng Việt.
- **Truy xuất**: BM25 tự cài đặt, semantic search bằng NumPy, hợp nhất thứ hạng bằng RRF và tái xếp hạng bằng Cross-Encoder `AITeamVN/Vietnamese_Reranker`.
- **Giao diện thử nghiệm**: Gradio và FastAPI.
- **Đánh giá**: pipeline RAGAS trong thư mục `src/evaluation`.

4.1.2 Dữ liệu thực nghiệm

Nguồn dữ liệu tri thức được xây dựng từ sách giáo khoa Tin học THPT lớp 10, 11 và 12 thuộc hai bộ Cánh Diều và Kết Nối Tri Thức. Dữ liệu gốc được chuyển đổi sang Markdown, làm sạch và phân mảnh theo cấu trúc phân cấp của sách.

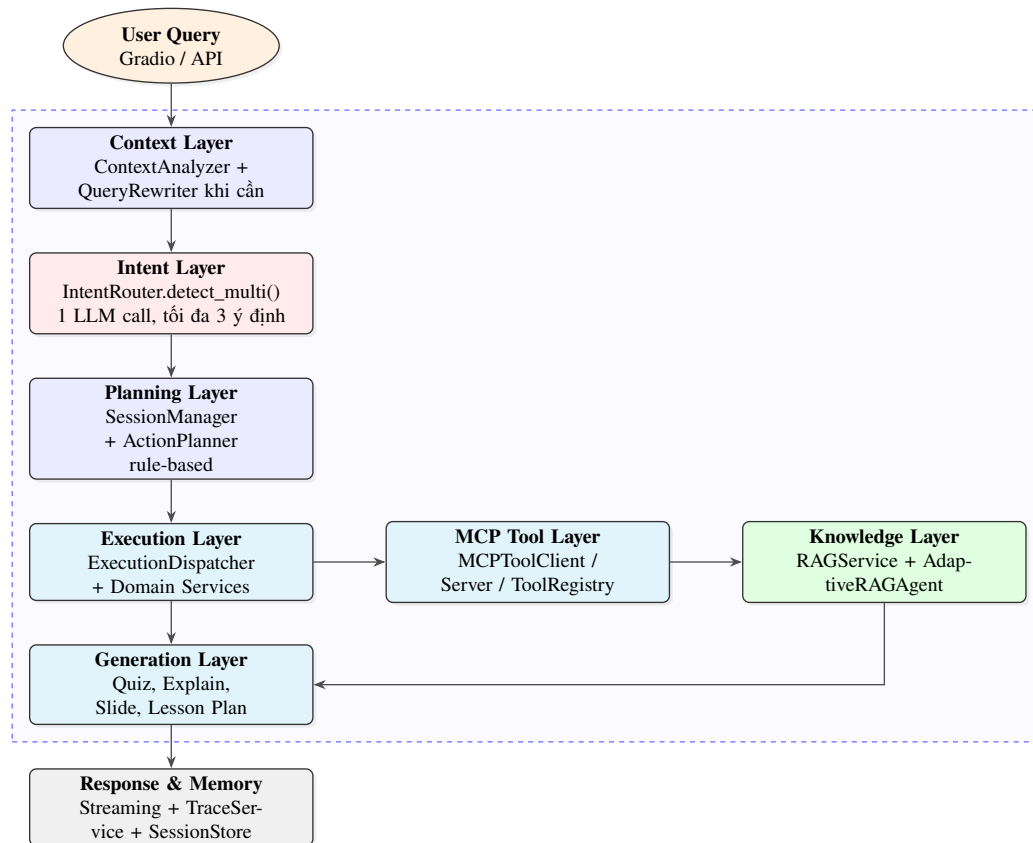
Kết quả chuẩn bị dữ liệu hiện tại:

- 6 tệp SGK đã làm sạch trong thư mục `rawdata`.
- 2.348 chunk có cấu trúc trong `data/rag_chunks_v2.json`.
- Ma trận embedding trong `data/embeddings.npy` với kích thước 2348×768 .
- Metadata của mỗi chunk gồm bộ sách, lớp, chủ đề, bài học, cấp phân cấp và loại nội dung.

4.2 Tổng quan kiến trúc hệ thống hiện tại

Kiến trúc hiện tại được chia thành hai lớp điều phối khác nhau. Luồng hỏi đáp, sinh câu hỏi, chấm điểm và quản lý phiên đi qua `Orchestrator` tự cài đặt bằng Python. Riêng luồng sinh slide và giáo án sử dụng một *content supervisor graph* dựa trên `LangGraph`, vì đây là tác vụ dài, có nhiều công cụ con và có bước duyệt dần ý bởi người dùng.

Sơ đồ tổng quan được thể hiện tại Hình ??.



Hình 4.1: Sơ đồ tổng quan kiến trúc hệ thống hiện tại

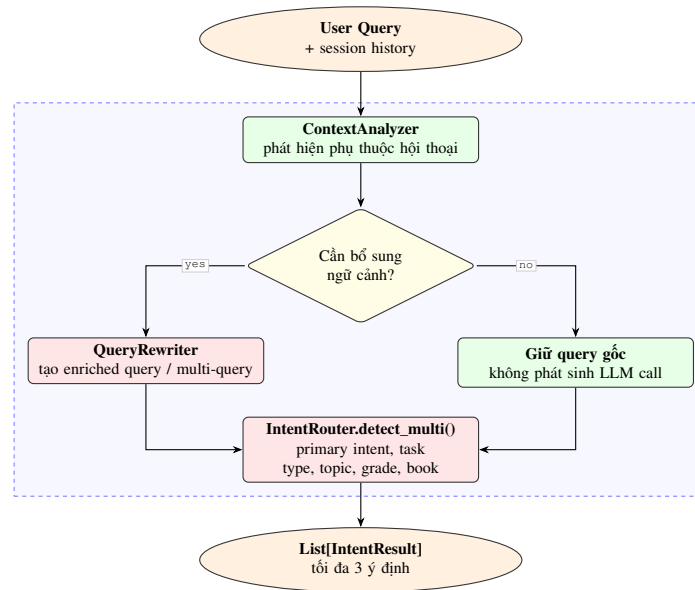
Trong kiến trúc này, `Orchestrator` chỉ giữ vai trò điều phối mỏng. Nó không chứa logic nghiệp vụ chi tiết mà chuyển việc thực thi sang `QuizService`, `SlideService`, `ExplainHandler`, `ChatHandler` hoặc các handler chuyên biệt. Phần truy xuất tri thức được gọi qua lớp MCP nội bộ, giúp tách domain service khỏi chi tiết cài đặt của RAG.

4.3 Thiết kế chi tiết các thành phần đã hiện thực

4.3.1 Lớp ngữ cảnh và nhận diện ý định

Lớp đầu vào gồm ba thành phần: `ContextAnalyzer`, `QueryRewriter` và `IntentRouter`. `ContextAnalyzer` là logic Python thuần, dùng để phát hiện các câu hỏi phụ thuộc ngữ cảnh như “cái này”, “bài đó”, hoặc các câu hỏi

tiếp nối. QueryRewriter chỉ được gọi khi cần mở rộng truy vấn cho RAG. IntentRouter là điểm gọi LLM chính để nhận diện danh sách ý định.

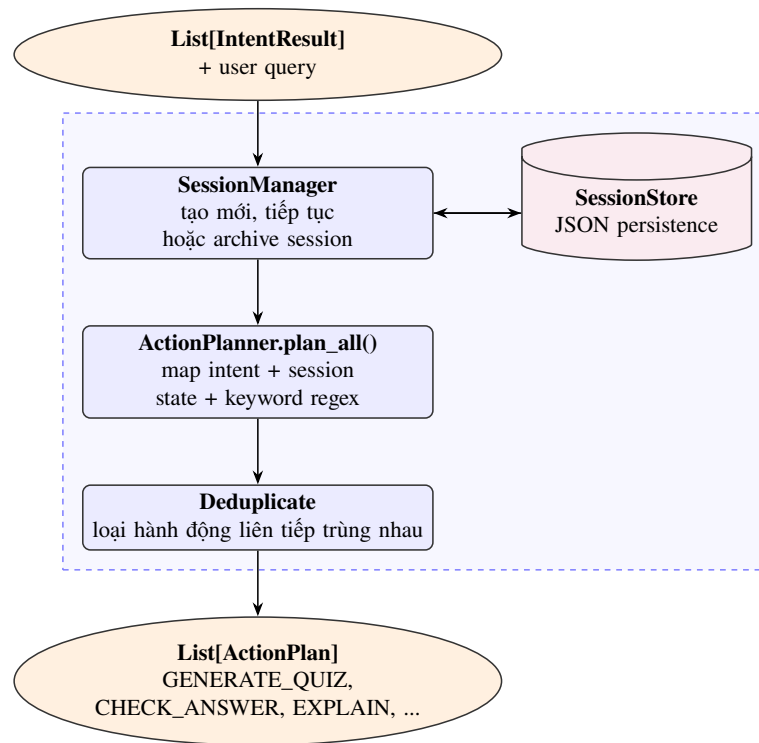


Hình 4.2: Luồng xử lý ngữ cảnh và nhận diện đa ý định

Đầu ra của lớp này là danh sách `IntentResult`. Mỗi phần tử chứa các trường chính như `primary_intent`, `task_type`, `topic`, `grade`, `book` và cờ `is_new_topic`. Thiết kế này cho phép một câu hỏi phức hợp như vừa yêu cầu giải thích, vừa tạo câu hỏi luyện tập được phân rã thành nhiều hành động độc lập ở các bước sau.

4.3.2 Lớp lập kế hoạch và quản lý phiên

Sau khi có ý định, hệ thống chuyển sang lớp lập kế hoạch. Hai thành phần chính là `SessionManager` và `ActionPlanner`. Đây là các module rule-based, không gọi LLM.

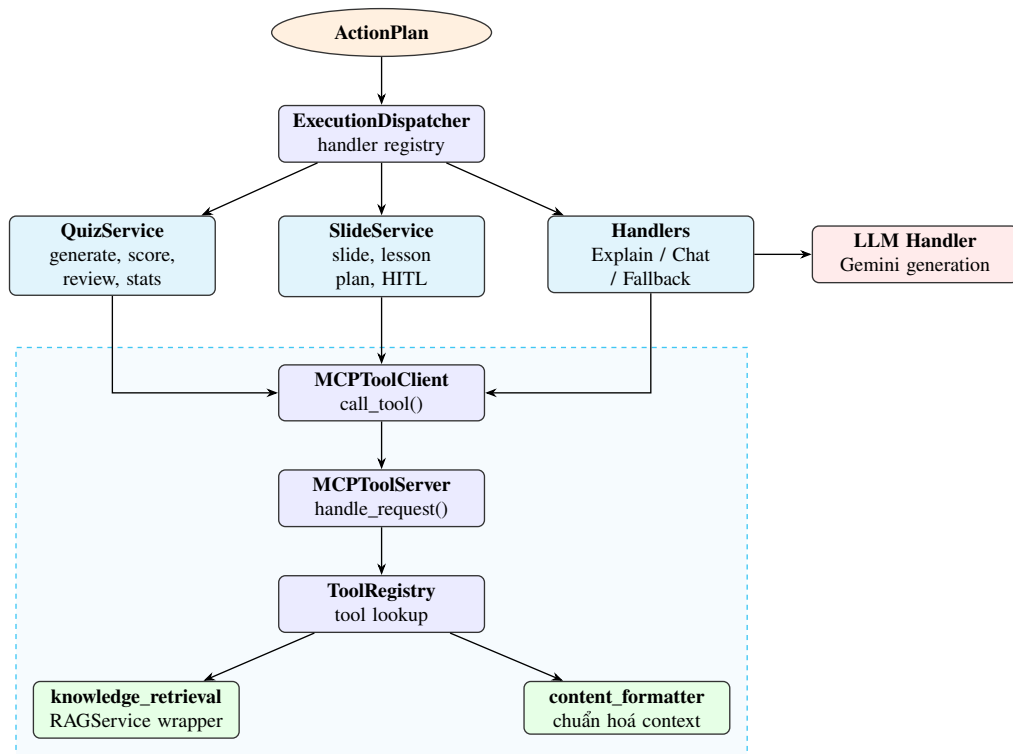


Hình 4.3: Lớp lập kế hoạch và duy trì trạng thái phiên

ActionPlanner hiện ánh xạ sang các hành động: sinh câu hỏi, sinh slide, sinh giáo án, chấm câu trả lời, ôn lại câu sai, xem thống kê, giải thích câu hỏi, trả lời bài tập trong slide, giải thích khái niệm và trò chuyện tự do. Việc tách bước này khỏi LLM giúp luồng điều phối ổn định hơn và dễ kiểm thử hơn.

4.3.3 Lớp thực thi và MCP Tool Layer

Lớp thực thi dùng `ExecutionDispatcher` để gọi đúng dịch vụ nghiệp vụ. Với các hành động cần tri thức SGK, dịch vụ không gọi trực tiếp RAG mà gọi qua `MCPToolClient`. Phía dưới là `MCPToolServer` và `ToolRegistry`. Trong mã nguồn hiện tại, Orchestrator đăng ký hai tool đang hoạt động là `knowledge_retrieval` và `content_formatter`. File `web_search_tool.py` đang tồn tại ở mức khung mở rộng, chưa được đăng ký trong Orchestrator hiện tại.

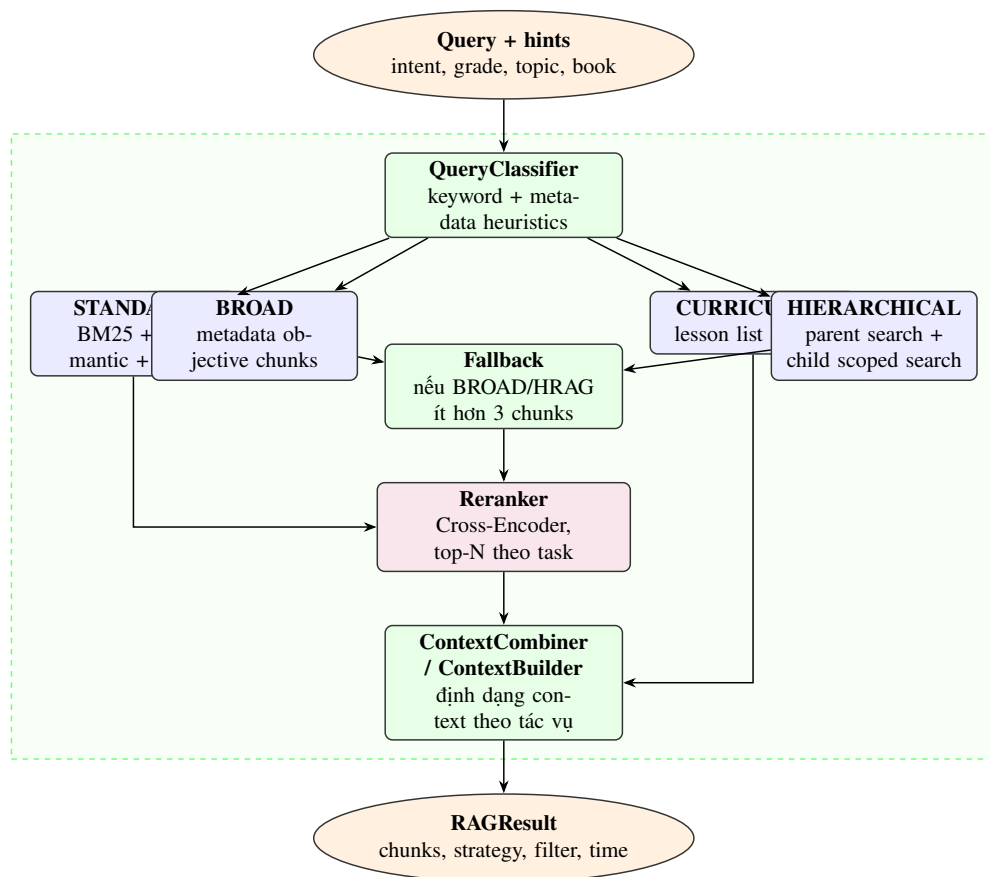


Hình 4.4: Lớp thực thi và cơ chế gọi công cụ qua MCP nội bộ

Thiết kế MCP nội bộ giúp các dịch vụ nghiệp vụ phụ thuộc vào interface công cụ thay vì phụ thuộc trực tiếp vào lớp RAG. Đây là điểm thay đổi quan trọng so với bản thảo cũ, vì nó làm rõ ranh giới giữa điều phối nghiệp vụ và năng lực truy xuất tri thức.

4.3.4 Knowledge Layer: Adaptive RAG

Phân hệ RAG hiện được đóng gói bởi RAGService và AdaptiveRAGAgent. Bộ phân loại QueryClassifier chọn chiến lược bằng heuristic, không dùng LLM. Các tín hiệu đầu vào gồm câu hỏi, intent hint, task type, lớp, chủ đề và bộ sách.



Hình 4.5: Knowledge Layer với Adaptive RAG bốn chiến lược

Bảng ?? tóm tắt mục đích của từng chiến lược.

Chiến lược	Mục đích sử dụng
STANDARD	Tìm kiếm phẳng bằng BM25 + semantic search, hợp nhất RRF, sau đó rerank.
BROAD	Lấy các chunk dạng mục tiêu/tổng quan khi người dùng hỏi khái quát về lớp, chủ đề hoặc bài học.
CURRICULUM	Trả về danh sách chủ đề, bài học hoặc cấu trúc chương trình bằng metadata lookup.
HIERARCHICAL	Truy xuất hai pha: tìm parent chunk cấp chủ đề/bài học, sau đó tìm child chunk chi tiết trong phạm vi đã khoanh.

Bảng 4.1: Các chiến lược truy xuất của Adaptive RAG

4.3.5 Generation Layer cho quiz, giải thích, slide và giáo án

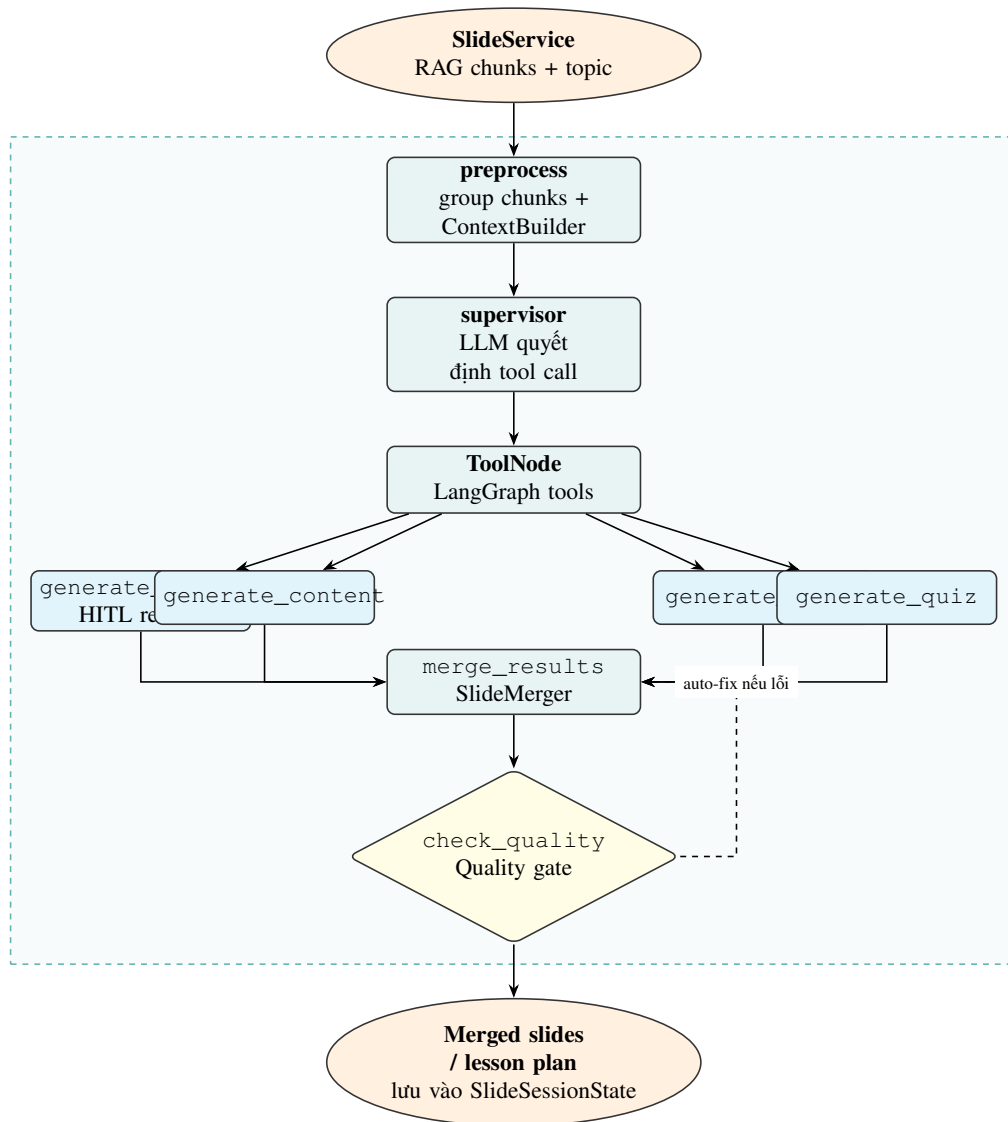
Lớp sinh nội dung không còn là một “validator agent” chung cho mọi nhánh như bản cũ. Trong mã nguồn hiện tại, các nhánh được tách theo dịch vụ:

- QuizService: sinh câu hỏi, kiểm duyệt bằng QuestionValidator,

lưu vào QuizRound, chấm điểm bằng QuestionScorer.

- ExplainHandler: sinh câu trả lời giải thích dựa trên context từ RAG.
- SlideService: chạy content supervisor graph để tạo slide hoặc giáo án.
- ChatHandler: trả lời hội thoại tự do, không bắt buộc đi qua RAG.

Riêng với slide và giáo án, hệ thống dùng LangGraph vì đây là workflow dài, cần nhiều bước công cụ và có human-in-the-loop ở bước duyệt outline.

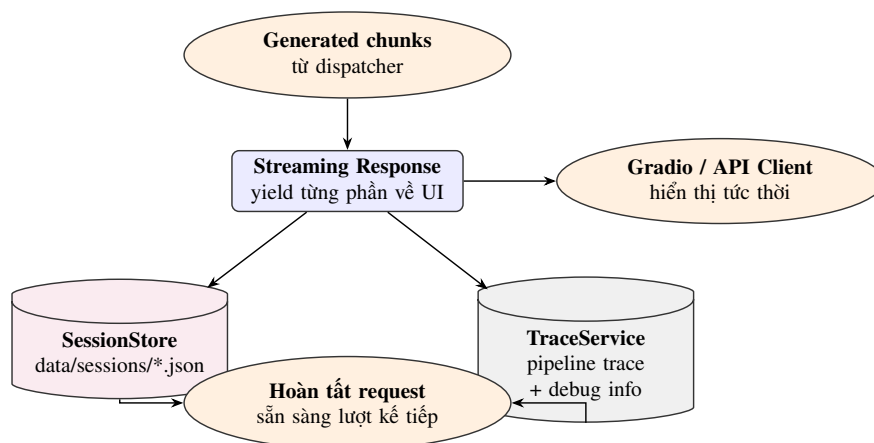


Hình 4.6: Content supervisor graph cho luồng sinh slide và giáo án

Với câu hỏi luyện tập, hệ thống dùng một nhánh khác. Câu hỏi do handler sinh ra phải đi qua QuestionValidator; chỉ các câu được duyệt mới được lưu vào session. Khi người học trả lời, QuestionScorer cập nhật kết quả vào QuestionRecord và StudentProfile.

4.3.6 Response, memory và logging

Chặng cuối của request lifecycle trả kết quả theo dạng streaming, lưu phiên làm việc và ghi trace. Trạng thái phiên gồm hội thoại, quiz rounds, câu trả lời của người học, slide output và các câu hỏi bài tập trong slide.



Hình 4.7: Lớp phản hồi, ghi nhớ và logging

4.4 Đánh giá hiệu năng và kết quả thực nghiệm

4.4.1 Phương pháp đánh giá

Đánh giá định lượng hiện tập trung vào phân hệ RAG. Tập kiểm thử gồm 246 mẫu câu hỏi - đáp án tham chiếu được tạo từ kho tri thức SGK. Pipeline đánh giá sử dụng RAGAS với bốn độ đo: Faithfulness, Answer Relevancy, Context Precision và Context Recall.

4.4.2 Kết quả RAGAS

Chỉ số đánh giá	Điểm trung bình
Faithfulness	0.9757
Answer Relevancy	0.8571
Context Precision with Reference	0.8555
Context Recall	0.9593

Bảng 4.2: Kết quả đánh giá RAGAS trên 246 mẫu

Faithfulness đạt 0.9757 cho thấy câu trả lời bám sát tốt vào ngữ cảnh truy xuất được. Context Recall đạt 0.9593, phản ánh khả năng tìm đủ thông tin cần thiết từ kho tri thức. Hai chỉ số Answer Relevancy và Context Precision đạt khoảng 0.85, cho thấy hệ thống đã trả lời đúng trọng tâm ở đa số trường hợp nhưng vẫn còn dư địa tối ưu ở bước xếp hạng và cô đọng context.

4.4.3 Đánh giá độ trễ

Thành phần	Thời gian trung bình
Retriever và Reranker	4.579 giây
Generator LLM	1.979 giây
Tổng pipeline	6.558 giây

Bảng 4.3: Thống kê độ trễ trung bình của pipeline RAG

Độ trễ chủ yếu nằm ở khâu truy xuất và tái xếp hạng, vì hệ thống phải thực hiện BM25, semantic search, RRF và Cross-Encoder reranking. Với quy mô 2.348 chunk, cách cài đặt bằng NumPy vẫn phù hợp cho prototype vì đơn giản, dễ kiểm soát và không cần phụ thuộc FAISS. Tuy nhiên, nếu mở rộng dữ liệu hoặc số người dùng đồng thời, cần bổ sung cache embedding truy vấn, giảm số lượng ứng viên đưa vào reranker hoặc chuyển sang vector index chuyên dụng.

4.5 Kiểm thử chức năng và trạng thái hoàn thiện

Các chức năng cốt lõi đã được hiện thực gồm: hỏi đáp dựa trên SGK, sinh câu hỏi bốn dạng, kiểm duyệt câu hỏi, chấm điểm câu trả lời, ôn tập câu sai, thống kê học tập trong phiên, sinh slide, sinh giáo án và lưu trạng thái phiên. Bảng ?? tóm tắt trạng thái hiện tại.

Nhóm chức năng	Trạng thái hiện tại
Hỏi đáp và giải thích kiến thức	Đã có <code>ExplainHandler</code> , truy xuất tri thức qua <code>knowledge_retrieval</code> .
Sinh câu hỏi	Đã hỗ trợ MCQ, tự luận, điền khuyết, đúng/sai.
Kiểm duyệt câu hỏi	Đã có <code>QuestionValidator</code> , lọc câu hỏi trước khi lưu vào phiên.
Chấm điểm và ôn tập	Đã có <code>QuestionScorer</code> , <code>QuizRound</code> , review câu sai và thống kê tiến độ.
Sinh slide/giáo án	Đã có <code>SlideService</code> và <code>content supervisor graph</code> với <code>outline</code> , <code>content</code> , <code>media</code> , <code>quiz</code> , <code>merge</code> , <code>quality gate</code> .
Lưu phiên và trace	Đã có <code>SessionStore</code> và <code>TraceService</code> .
Web search qua MCP	Chưa phải luồng active trong <code>Orchestrator</code> hiện tại; file tool còn ở mức khung mở rộng.

Bảng 4.4: Trạng thái các nhóm chức năng trong codebase hiện tại

Hạn chế hiện tại là đánh giá định lượng mới tập trung vào RAG, chưa có bộ đo

riêng cho chất lượng câu hỏi, giáo án và slide. Luồng sinh slide/giáo án cũng cần thêm kiểm thử thực tế trên nhiều bài học để đánh giá độ ổn định của HITL, merge và quality gate.

4.6 Kết chương

Chương này đã cập nhật lại phần hiện thực và thực nghiệm theo trạng thái code-base hiện tại. Hệ thống đã chuyển trọng tâm từ mô tả agent tổng quát sang kiến trúc điều phối rõ ràng: Orchestrator Python thuần cho request lifecycle, MCP nội bộ cho công cụ, Adaptive RAG cho truy xuất tri thức và content supervisor graph cho các tác vụ sinh học liệu dài. Các kết quả RAGAS cho thấy phân hệ truy xuất - sinh trả lời đạt độ bám sát tài liệu cao, đồng thời chỉ ra các hướng tối ưu tiếp theo về precision, latency và đánh giá chất lượng học liệu.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương này trình bày chi tiết những đóng góp nổi bật nhất, bao gồm các kiến trúc, giải thuật và mô hình thiết kế được tác giả tự xây dựng từ đầu (from scratch) nhằm giải quyết các bài toán khắt khe trong hệ thống Trợ lý ảo Giáo dục. Thay vì phụ thuộc vào các nền tảng đóng gói sẵn của bên thứ ba, tôi đã phân tích và thiết kế lại hàng loạt các cụm tính toán nhằm giải quyết các điểm nghẽn hiệu năng, hạn chế sự ảo giác thông tin và tối ưu hóa luồng thực thi trong môi trường đòi hỏi độ chính xác tuyệt đối như giáo dục học thuật.

5.1 Tự xây dựng bộ máy truy xuất kết hợp (BM25 + Semantic + RRF) từ mức cơ sở

5.1.1 Dẫn dắt vấn đề

Trong hệ thống RAG, để mô hình có thể bám sát nội dung sách giáo khoa, hệ thống cần khả năng truy xuất đồng thời theo từ khóa chuyên ngành chính xác (Keyword-based) và theo ngữ nghĩa tương đồng (Semantic-based). Giải pháp biểu diễn từ khóa truyền thống như TF-IDF thường gặp hạn chế nghiêm trọng do thiên lệch độ dài văn bản (text length bias), khiến các phần nội dung dài dễ dàng lấn át các định nghĩa ngắn gọn - vốn rất phổ biến trong sách giáo khoa - dù mức độ liên quan của chúng không cao.

Ở một khía cạnh khác, việc ứng dụng ngay các bộ máy tìm kiếm công nghiệp công kênh như Elasticsearch hoặc thư viện nền tảng như FAISS lại đặt ra sự hao phí tài nguyên không cần thiết. Tập dữ liệu cấu trúc sách giáo khoa của dự án chỉ dừng lại ở phạm vi gần 2348 khối văn bản phân mảnh tĩnh. Hơn nữa, những thư viện truy xuất vector cục bộ như FAISS không hỗ trợ sẵn (native support) thuật toán BM25 gốc, gây ra nhiều khó khăn trong việc hợp nhất.

5.1.2 Giải pháp đề xuất

Thay vì phụ thuộc thư viện, tôi đã tự lập trình lại thuật toán BM25 và khung tìm kiếm ngữ nghĩa độc lập. Thuật toán BM25 được cấy nội bộ với hai tham số kiểm soát $k_1 = 1.2$ và $b = 0.75$. Tham số k_1 khống chế mức độ bão hòa sự xuất hiện của từ khóa, trong khi b điều phối sự tinh duyệt, vừa đủ để phạt bớt các văn bản quá dài mà không tàn phá các đoạn định lý học thuật quan trọng cấu trúc nguyên vẹn.

Về tìm kiếm ngữ nghĩa, luồng thực thi sử dụng mô hình nhúng `dangvantuan/vietnamese-document-embedding` hỗ trợ vectơ 768 chiều tối ưu hóa trực tiếp cho tiếng Việt. Các vectơ biểu diễn văn bản (embeddings) được trích xuất hoàn toàn khả lập trước (pre-computed). Tôi áp dụng thêm phép chuẩn hóa L2 lên hệ vectơ, giúp việc

tính toán độ đo khoảng cách (Cosine Similarity) cho không gian người dùng tra cứu chỉ quy lại thành những phép nhân ma trận dấu chấm vô cùng giản đơn.

Khâu kết hợp 2 luồng điểm số truy cập này lại với nhau được định nghĩa bằng kỹ thuật Dung hợp Hạng nghịch đảo (Reciprocal Rank Fusion - RRF). Công thức RRF được tùy biến cấu hình tại lớp code với hằng số $k = 60$. Hằng số mượt này đã được tôi phân tích cẩn thận, nó khóa chặn yếu điểm đột biến bảng xếp hạng (rank bias), tạo ra sự hòa quyện tuyệt đối giữa hai dạng truy xuất nhằm đảm bảo không một kỹ thuật nào được phép chiếm ưu thế hoàn toàn so với kỹ thuật còn lại.

5.1.3 Kết quả đạt được

Sự đánh đổi công sức lập trình này tạo ra kết quả vô cùng tuyệt vời. Mã nguồn vận hành siêu nhẹ (lightweight), hoàn toàn tương thích và dễ gỡ lỗi do mọi quá trình sinh điểm thứ hạng đều tường minh. Hệ thống tự triệt tiêu đi các độ trễ gọi dịch vụ cơ sở dữ liệu bên ngoài, trong khi vẫn đạt được tính thống nhất cả về độ chính xác của từ khóa đặc thù chuyên ngành lẫn mức độ thông hiểu ý nghĩa ngữ cảnh học bài.

5.2 Thiết kế Điều phối viên mức mã nguồn (Code-level Orchestrator)

5.2.1 Dẫn dắt vấn đề

Khuynh hướng hiện đại khi xây dựng chuỗi thao tác LLM thường lạm dụng các khung framework có sẵn như LangChain, LlamaIndex hay LangGraph. Sự mượt mà tiện dụng của chúng đi kèm theo vô vàn đoạn mô đun ẩn (opaque flow/magic abstractions) làm cho dấu vết dịch chuyển chuỗi dữ liệu rất mờ mịt. Khi xảy ra sai sót dữ liệu do lệnh prompt sinh ra từ thư viện mâu thuẫn hệ thống, chuỗi khối này gây trở lại rào cản chết (heavy dependency), làm việc gỡ lỗi trở thành thảm họa.

5.2.2 Giải pháp đề xuất

Thay vì đóng hộp đồ án, tôi đã cấu trúc và tự thiết kế một *Orchestrator* (Điều phối viên tác vụ) hoàn toàn tại lớp mức lệnh cơ sở gốc (pure-code nodes). Orchestrator được định hình là "Thin controller" (Bộ xử lý màng xấp xỉ), nó điều phối dòng xử lý xoay quanh bốn hàm tự nhiên: `ContextAnalyzer`, `SessionManager`, `ActionPlanner` và `RAG Agent` mà không cần nhúng LangChain.

Ý tưởng chủ đạo ở đây là dồn tất cả tiến trình gọi LLM đắt đỏ (LLM call) về mức tính tối giản: Hệ thống bắt buộc dùng LLM Call duy nhất tại lúc nhận diện (Intent Detection). Tất cả các luồng trung chuyển dữ liệu ở tầng `ActionPlanner`, việc nối context từ bộ đệm... đều được tính toán bằng Python nguyên thủy cùng các quy luật cây trạng thái. Không có mô hình ngôn ngữ nào tham gia cản trở ở bước điều hướng, đảm bảo luồng đi luôn an toàn, không có ảo giác mã rẽ.

5.2.3 Kết quả đạt được

Mô hình Orchestrator này giúp toàn bộ kết cấu độ trễ (latency) của toàn chuỗi dịch vụ bị triệt tiêu, trả lại tốc độ phản hồi gần mức API nguyên chất thuần túy. Quá trình theo vết vòng đời câu hỏi thông qua Console Logging (nhằm quan sát tham số mỗi nhử hay số block ngữ cảnh nạp vào) cực kỳ dễ dàng. Tuy sự thay thế framework công nghiệp đồng nghĩa với việc đòi hỏi sinh viên phải chịu sức ép mở rộng hệ đa tác vụ thủ công trong tương lai, sự bám rễ linh hoạt này hoàn toàn vô cấp khóa nền tảng (zero vendor lock-in).

5.3 Kiến trúc Adaptive RAG Agent với suy luận chọn lọc Heuristics

5.3.1 Dẫn dắt vấn đề

Một quy trình xử lý khối dữ liệu (RAG pipeline) chung không thể bao phủ nổi các phong cách tương tác biến hóa. Sẽ là lãng phí nếu đem quy trình gom tập RAG đi lọc toàn bộ một tệp thư viện lớn khi đối phương chỉ hỏi cách truy cập cấu trúc phân tập (Curriculum) theo sách. Nhưng nếu lạm dụng LLM vào vai trò phán đoán "Tôi nên truy xuất kiểu gì vào lúc này", hệ thống một lần nữa sẽ tắc nghẽn về tốc độ và chi phí hoạt động (cost latency).

5.3.2 Giải pháp đề xuất

Tôi đã xây dựng thành phần *Adaptive RAG Agent* hỗ trợ cho luồng đi nội tại và không giao nhiệm vụ quyết định cho LLM, thay vào đó là việc khai thác khả năng của thuật toán dựa trên suy nghiệm (Heuristics-based *QueryClassifier*).

Bộ phân loại quy tắc truy xuất dựa trên khớp từ vựng chuẩn (keyword matching) phân tích chủ đề định nghĩa (topic hints) từ lịch sử chat, xác định bài toán trong mức dưới 1/100 giây thông qua các tập Regular Expression để dẫn dắt đến một trong bốn chiến lược sau:

- **STANDARD (Chuẩn):** Tập kết hoàn chỉnh luồng BM25 kết hợp Semantic Fusion RRF và sau đó là Reranker phục vụ khai phá câu hỏi học sâu đa hướng.
- **BROAD (Rộng):** Áp dụng luồng xử lý thu mục tiêu tổng quát (metadata lookup for objective chunks), bỏ hẳn quá trình Xếp hạng lại (Reranking). Cửa ngõ này đáp ứng các câu tổng hợp mô tả mà giúp tốc độ truy vấn tăng xấp xỉ 10 lần.
- **CURRICULUM (Mục lục):** Thu gọn phạm vi tra cứu trên tập tin đồ hệ thống sách tạo dàn ý.
- **HRAG (Hierarchical):** Mở bối cảnh theo hai cấp (Thô và Tinh), phục vụ riêng cho các phân đoạn định nghĩa phức tạp đa tính liên đới bài tập chéo.

Để giải quyết tính cố định yếu điểm của quy luật, một tập thuật điều phối lùi (fallback logic) đóng vai trò lá chắn. Ví dụ, nếu luồng BROAD lọc dữ liệu không xuất trả ra đủ số lượng mảnh ngữ nghĩa (dưới 3 chunks), hệ thống tự kích hoạt quay ngược luồng tiêu chuẩn STANDARD nhằm bảo vệ an toàn bối cảnh trước khi chuyển đi.

5.3.3 Kết quả đạt được

Áp dụng cơ chế Heuristic giảm tải quá tải hệ thống khi phải thực hiện hàng chục mô hình truy thu, nhưng vẫn giúp LLM sinh đáp án với nguồn cấp dữ liệu (grounding) chuẩn nhất.

5.4 Nhận diện Ý định Hai Cấp (2-Level Intent Detection) giải tỏa độ trễ

5.4.1 Dẫn dắt vấn đề

Lớp tiếp điểm nhận giao tiếp tự nhiên đòi hỏi việc xác định luồng tác vụ. Nếu sử dụng LLM tạo kịch bản xử lý tự lập hoàn toàn theo dạng Agent pattern vòng lặp (gọi LLM liên tiếp tạo ý định, gọi tiếp LLM phân tách bài, tiếp tục gọi LLM lập action plan), một chu trình đính kèm tốn kém tối thiểu 3 đến 5 API requests dẫn đến hàng chục giây độ trễ rập khuôn.

5.4.2 Giải pháp đề xuất

Thiết kế đột phá được cấy vào hệ thống bằng chiến lược **Nhận diện ý định Hai cấp (2-Level Intent Detection)**. Tại Cấp độ 1, kỹ thuật đẩy một lời gọi LLM duy nhất với mức kiểm soát tham số sáng tạo ngắt chặt ở mức nhiệt *temperature* = 0.1. Chỉ dẫn (instruction) ép buộc hệ thống bóc tách hoàn chỉnh các siêu thông tin chứa: Tập nhóm Lệnh (intent), Loại nhiệm vụ (task type), Khoảng nội dung (topic), Lớp học và Đầu sách. Nội dung JSON trả về ép nhận diện song song cho nhiều ý định gõ trong cùng một thời điểm, tuy nhiên tôi chốt giới hạn ngưỡng (Max 3 Intents) để chặn đứt mọi hậu quả suy diễn lây lan mạng (state explosion).

Tại Cấp độ 2, Module `ActionPlanner` khởi động với luồng dữ liệu truyền vào là mã tĩnh không chế, chuyển chuỗi dịch tham chiếu từ JSON List sang mã lệnh vận hành Enum nguyên khối thông thường.

5.4.3 Kết quả đạt được

Bằng cấu trúc này, đồ án đã tách ghép sự tự trị quá đáng của hệ mạng đa tác nhân. Phương châm thiết kế “1 lượt LLM bóc tách gốc rễ kết hợp 0 lượt LLM vận hành lệnh phân phối” định hình trơn tru hiệu suất, cắt đứt áp lực độ trễ đứt gãy mà thông số độ ổn định hệ thống vẫn cực kì tuyệt đối.

5.5 Hệ thống Phản biện khép kín (Self-Reflection bằng QuestionValidator)

5.5.1 Dẫn dắt vấn đề

LLM dễ bị tấn công suy diễn bất biến (hallucination) ở bất kể phiên bản năng lực mạnh thế nào, đặc biệt khi được giao những cấu trúc tạo khuôn ra bộ kiểm tra trắc nghiệm (Quiz JSON Arrays) hay bố cục trình diễn. Sự lọt lưới một câu trả lời sai đi ngược quan điểm trong sách chuẩn sẽ biến hệ thống giáo dục thành mối nguy hiểm thảm họa.

5.5.2 Giải pháp đề xuất

Nền tảng sinh mã đã được tách ly làm hai để triển khai **Hệ thống Phản biện Tự động hai LLM (Dual-LLM Cross-check)**. Toàn bộ kết quả mảng câu hỏi trắc nghiệm sinh ra từ LLM thực thi sẽ không được xả thẳng lên mạng lưới trình duyệt, mà lập tức bị bắt lại tại Cổng kiểm duyệt `QuestionValidator`. Tác nhân ở cổng này không làm nhiệm vụ sinh đáp án, mà mang riêng System Prompt đóng vai giám thị, đối sánh cấu trúc nội dung từ khóa được tạo ra với các luật khắt khe trong Sách Giáo Khoa (Context Source) và Khung tiêu chí xếp dạng câu (Rubrics).

Nếu kết quả đối sánh đánh điểm không đạt cấu trúc, luồng sinh cấu trúc sẽ nhận một lời báo lỗi hồi quy tự suy sửa chữa (retry generation). Tách biệt vòng xoáy thời gian, quá trình này bị hệ thống gài lặp tối đa hai lượt.

5.5.3 Kết quả đạt được

Chỉ những nội dung mang phân định "Approved" lọt qua khe mới được gửi về giao diện. Tính hệ thống định hướng cấu trúc (Structured output limit) biến mọi phiên sinh của đề án thành tệp dữ liệu không góc lồi, nâng cấp tính ứng dụng của nó đến cấp hoạt động ổn định ở tính thương mại thực tế do dữ liệu trả về sở hữu tính căn bản an toàn.

5.6 Chiến lược Quản lý Trạng thái Bộ nhớ linh hoạt (Memory State Strategy)

5.6.1 Dẫn dắt vấn đề

Phiên bản Trợ lý ảo duy trì một phiên đàm thoại hỏi bài dài hàng giờ sẽ tích nập chuỗi tin nhắn rất lớn. Việc gom tổng lịch sử thô (full text history) vào luồng prompt mới để giúp LLM bắt mạch hội thoại vừa lãng phí Token chi phí quá đắt đỏ lại dễ làm loãng dữ kiện cốt lõi ở câu lệnh đầu do hiện trạng giới hạn sổ nhớ mô hình chặn đầu cản đuôi. Một giải pháp sử dụng cơ sở dữ liệu lớn để quản lý lại đánh mất tiêu chí gọn nhẹ linh hoạt.

5.6.2 Giải pháp đề xuất

Tôi đã áp dụng giải pháp quản trị *Windows-Context* (Khoanh vùng giới hạn) tích hợp trạng thái *Session* lưu vết *JSON* cục bộ. Cơ chế đếm cửa sổ chặn giới hạn

nap `max_messages=10` (chiết suất trượt đọc lưu 10 lượt giao tác mới nhất) điều hướng dữ liệu. Mạch liên kết không bị tan rã, token nội dung không phình to nhờ nhét dẫn đến các luồng rác. Tuy giải quyết luồng trượt nhưng bảo toàn bền vững phiên, mỗi người dùng gõ câu lệnh đều lưu tệp sao lưu phiên dạng file định danh tính đĩa cứng riêng.

Bảng mạch đặc biệt dành riêng cho tính năng báo sửa luyện thi rẽ nhánh bằng khối `QuizRound`. Phân đệm tách rời tập ghi chú dữ liệu lịch sử hội thoại chuẩn giúp học sinh có thể yêu cầu Trợ lý lục soát lại “Câu sai môn đó lúc nãy” mà nội dung này không hề nhồi đệm xen lẫn vào hệ thống lưu trữ giao tiếp thông thường.

5.6.3 Kết quả đạt được

Chiến lược đem đến thiết kế kiến trúc theo cơ sở vô trạng thái (Stateless server), nơi RAM hệ thống không bị tràn do tiêu hủy trạng thái tự nhiên, bù lấp bằng truy xuất tập tệp chuẩn rất nhẹ cục bộ. Sự tách gỡ này không chỉ tinh giản kiến trúc xử lý vận hành tiết kiệm chi phí mà làm bàn đạp để tinh chỉnh sau này mở rộng quy mô đổ xuống DB lưu trữ dữ liệu tập trung.

5.7 Thiết kế Hệ thống Đa tác nhân (Multi-Agent System) cho quá trình Sinh Slide Tự động

5.7.1 Tổng quan kiến trúc: Mô hình Supervisor (Supervisor Pattern) và Lựa chọn Sub-Agent

Nhu cầu tự động hóa việc tạo bài giảng (Slide) đòi hỏi hệ thống phải xử lý lượng thông tin đồ sộ từ việc trích xuất kiến thức, tóm tắt, tìm kiếm hình ảnh minh họa, cho đến việc sinh câu hỏi củng cố. Nếu gom tất cả các nhiệm vụ này vào một Agent duy nhất (Monolithic Agent), ngữ cảnh truyền vào (context window) từ RAG, kế hoạch (plan) và nội dung sẽ trở nên khổng lồ. Điều này dẫn đến hiện tượng quên lửng giữa chừng (Lost in the middle) khiến mô hình mất tập trung, không định hướng được các nhiệm vụ rời rạc.

Để giải quyết bài toán trên, tôi đã áp dụng mô hình **Supervisor Pattern**. Trong kiến trúc này, một nút chỉ huy (Supervisor) đóng vai trò trung tâm tiếp nhận yêu cầu và định tuyến tuần tự đến các tác nhân con (Sub-Agents) chuyên biệt. Mô hình này mang lại khả năng kiểm soát luồng chặt chẽ và rất thuận tiện để mở rộng hay nâng cấp từng thành phần độc lập. Các Sub-Agent được cân nhắc dựa trên tiến trình sơ phạm thực tế của một bài giảng chuẩn mực, phân tách quy trình ra làm 5 Sub-Agent xử lý theo trình tự chu trình làm việc (workflow) dưới đây.

5.7.2 OutlineAgent: Thiết lập Dàn ý Tổng quát

Nhiệm vụ: Tạo dàn ý (outline) tổng quát cho toàn bộ slide bài giảng.

Lý do tách biệt: OutlineAgent chỉ tập trung trả lời câu hỏi "Bài giảng này nên có những slide nào, tiêu đề là gì?". Việc phân tách đầu tiên này cắt giảm được luồng tư duy phức tạp của ngôn ngữ, giúp mô hình trở nên nhẹ hơn rất nhiều, bảo đảm cấu trúc đầu ra tuân thủ đúng mạch logic cốt lõi.

5.7.3 ContentAgent: Phát triển Nội dung Chi tiết (Cơ chế Fan-out)

Nhiệm vụ: Dựa trên bộ dàn ý có sẵn được trích xuất từ OutlineAgent, ContentAgent tiến hành đắp thịt bằng việc sinh nội dung chi tiết cho từng slide.

Cơ chế Fan-out (Phân tán): Các node xử lý được thiết kế độc lập, không phụ thuộc vào chuỗi tuần tự đăng trước hoặc đăng sau. ContentAgent khởi tạo luồng song song đắp thông tin cho từng chunk dữ liệu slide độc lập, giúp tốc độ sinh bài giảng được đẩy nhanh lên tối đa.

5.7.4 MediaAgent: Tích hợp Dữ liệu Trực quan

Nhiệm vụ: Thu thập và gợi ý các hình ảnh, đồ họa dạng GIF hoạt hình minh họa cho bài giảng.

Lý do cần thiết: Bất kỳ một bài giảng sư phạm tốt nào cũng cần có phần khởi động (hook) kích thích thị giác, hoặc chèn các ví dụ hình ảnh sinh động giúp truyền đạt những khái niệm trừu tượng một cách tổng quan và dễ hiểu. Việc này đánh mạnh vào tính định hướng tương tác (engagement) cho người học trong hệ thống giáo dục thay vì những khối văn bản khô khan.

5.7.5 QuizAgent: Sinh bài tập Hệ thống hóa Kiến thức

Nhiệm vụ: Tự động sinh ra bộ câu hỏi luyện tập trực tiếp đặt ở cuối bài giảng.

Lý do cần thiết: QuizAgent được tái sử dụng hoàn toàn (reuse code) từ dịch vụ QuizService cốt lõi đã có trước đó. Trong các bài giảng giáo dục, luôn cần tồn tại phần củng cố kiến thức lúc chốt bài. QuizAgent tự động chèn phần ôn tập cuối giờ này, giúp gia tăng giá trị trọn gói của một tiết học tự động.

5.7.6 MergeAgent: Tổng hợp và Kiểm duyệt (Bắt buộc)

Nhiệm vụ: Tổng hợp lại kết quả chấp vá từ các luồng Outline, Content, Media và Quiz thành một khối bản trình chiếu duy nhất và hoàn chỉnh.

Cơ chế Fan-in (Hội tụ): Nút tác nhân này thiết lập cơ chế vòng chặn đợi tất cả các Sub-Agent trước đó xử lý xong, đóng vai trò như một Cổng bảo chứng chất lượng (Quality Gate) cuối cùng. Nó bắt buộc kiểm tra độ hoàn chỉnh của dữ liệu JSON chấp vá để chắc chắn không một slide hay hình ảnh, câu trắc nghiệm nào bị bỏ sót hay gây luồng trước khi gửi kết quả về cho hệ thống hiển thị phía người dùng.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Sinh viên so sánh kết quả nghiên cứu hoặc sản phẩm của mình với các nghiên cứu hoặc sản phẩm tương tự.

Sinh viên phân tích trong suốt quá trình thực hiện ĐATN, mình đã làm được gì, chưa làm được gì, các đóng góp nổi bật là gì, và tổng hợp những bài học kinh nghiệm rút ra nếu có.

6.2 Hướng phát triển

Trong phần này, sinh viên trình bày định hướng công việc trong tương lai để hoàn thiện sản phẩm hoặc nghiên cứu của mình.

Trước tiên, sinh viên trình bày các công việc cần thiết để hoàn thiện các chức năng/nhiệm vụ đã làm. Sau đó sinh viên phân tích các hướng đi mới cho phép cải thiện và nâng cấp các chức năng/nhiệm vụ đã làm.

MỘT SỐ LƯU Ý VỀ TÀI LIỆU THAM KHẢO

Lưu ý: Sinh viên không được đưa bài giảng/slide, các trang Wikipedia, hoặc các trang web thông thường làm tài liệu tham khảo.

Một trang web được phép dùng làm tài liệu tham khảo **chỉ khi** nó là công bố chính thống của cá nhân hoặc tổ chức nào đó. Ví dụ, trang web đặc tả ngôn ngữ XML của tổ chức W3C <https://www.w3.org/TR/2008/REC-xml-20081126/> là TLTK hợp lệ.

Có năm loại tài liệu tham khảo mà sinh viên phải tuân thủ đúng quy định về cách thức liệt kê thông tin như sau. Lưu ý: các phần văn bản trong cặp dấu < > dưới đây chỉ là hướng dẫn khai báo cho từng loại tài liệu tham khảo; sinh viên cần xóa các phần văn bản này trong ĐATN của mình.

<**Bài báo đăng trên tạp chí khoa học:** Tên tác giả, tên bài báo, tên tạp chí, volume, từ trang đến trang (nếu có), nhà xuất bản, năm xuất bản >

hovy1993automated E. H. Hovy, "Automated discourse generation using discourse structure relations," *Artificial intelligence*, vol. 63, no. 1-2, pp. 341–385, 1993

<**Sách:** Tên tác giả, tên sách, volume (nếu có), lần tái bản (nếu có), nhà xuất bản, năm xuất bản>

peterson2007computer L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.

NguyenThucHai N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.

<**Tập san Báo cáo Hội nghị Khoa học:** Tên tác giả, tên báo cáo, tên hội nghị, ngày (nếu có), địa điểm hội nghị, năm xuất bản>

poesio2001discourse M. Poesio and B. Di Eugenio, "Discourse structure and anaphoric accessibility," in *ESSLLI workshop on information structure, discourse structure and discourse semantics*, Copenhagen, Denmark, 2001, pp. 129–143.

<**Đồ án tốt nghiệp, Luận văn Thạc sĩ, Tiến sĩ:** Tên tác giả, tên đồ án/luận văn, loại đồ án/luận văn, tên trường, địa điểm, năm xuất bản>

knott1996data A. Knott, "A data-driven methodology for motivating a set of coherence relations," Ph.D. dissertation, The University of Edinburgh, UK, 1996.

<**Tài liệu tham khảo từ Internet:** Tên tác giả (nếu có), tựa đề, cơ quan (nếu

có), địa chỉ trang web, thời gian lần cuối truy cập trang web>

BernersTim T. Berners-Lee, *Hypertext transfer protocol (HTTP)*. [Online]. Available: <ftp://info.cern.ch/pub/www/doc/http-spec.txt>. Z (visited on 09/30/2010).

LectureA Princeton University, *Wordnet*. [Online]. Available: <http://www.cogsci.princeton.edu/~wn/index.shtml> (visited on 09/30/2010).

PHỤ LỤC

A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP

Quy định chung

Dưới đây là một số quy định và hướng dẫn viết đồ án tốt nghiệp mà bắt buộc sinh viên phải đọc kỹ và tuân thủ nghiêm ngặt.

Sinh viên cần đảm bảo tính thống nhất toàn báo cáo (font chữ, căn dòng hai bên, hình ảnh, bảng, margin trang, đánh số trang, v.v.). Để làm được như vậy, sinh viên chỉ cần sử dụng các định dạng theo đúng template DATN này. Khi paste nội dung văn bản từ tài liệu khác của mình, sinh viên cần chọn kiểu Copy là “Text Only” để định dạng văn bản của template không bị phá vỡ/vi phạm.

Tuyệt đối cấm sinh viên đạo văn. Sinh viên cần ghi rõ nguồn cho tất cả những gì không tự mình viết/vẽ lên, bao gồm các câu trích dẫn, các hình ảnh, bảng biểu, v.v. Khi bị phát hiện, sinh viên sẽ không được phép bảo vệ DATN.

Tất cả các hình vẽ, bảng biểu, công thức, và tài liệu tham khảo trong DATN nhất thiết phải được SV giải thích và tham chiếu tới ít nhất một lần. Không chấp nhận các trường hợp sinh viên đưa ra hình ảnh, bảng biểu tùy hứng và không có lời mô tả/giải thích nào.

Sinh viên tuyệt đối không trình bày DATN theo kiểu viết ý hoặc gạch đầu dòng. DATN không phải là một slide thuyết trình; khi người đọc không hiểu sẽ không có ai giải thích hộ. Sinh viên cần viết thành các đoạn văn và phân tích, diễn giải đầy đủ, rõ ràng. Câu văn cần đúng ngữ pháp, đầy đủ chủ ngữ, vị ngữ và các thành phần câu. Khi thực sự cần liệt kê, sinh viên nên liệt kê theo phong cách khoa học với các ký tự La Mã. Ví dụ, nhiều sinh viên luôn cảm thấy hối hận vì (i) chưa cố gắng hết mình, (ii) chưa sắp xếp thời gian học/chơi một cách hợp lý, (iii) chưa tìm được người yêu để chia sẻ quãng đời sinh viên vất vả, và (iv) viết DATN một cách cẩu thả.

Trong một số trường hợp nhất thiết phải dùng các bullet để liệt kê, sinh viên cần thống nhất Style cho toàn bộ các bullet các cấp mà mình sử dụng đến trong báo cáo. Nếu dùng bullet cấp 1 là hình tròn đen, toàn bộ báo cáo cần thống nhất cách dùng như vậy; ví dụ như sau:

- Đây là mục 1 – Thực sự không còn cách nào khác tôi mới dùng đến việc bullet trong báo cáo.
- Đây là mục 2 – Nghĩ lại thì tôi có thể không cần dùng bullet cũng được. Nên tôi sẽ xóa bullet và tổ chức lại hai mục này trong báo cáo của mình cho khoa

học hơn. Tôi muốn thầy cô và người đọc cảm nhận được tâm huyết của tôi trong từng trang báo cáo DATN.

A.1 Ngành học

Sinh viên lưu ý viết đúng ngành/chuyên ngành trên bìa và trên gáy theo đúng quy định của Trường. Ngành học hay chuyên ngành học phụ thuộc vào ngành học mà sinh viên đăng ký. Sinh viên có thể đăng nhập trên trang quản lý học tập của mình để xem lại chính xác ngành học của mình.

Một số ví dụ sinh viên có thể tham khảo dưới đây, trong trường hợp có chuyên ngành thì sinh viên không cần ghi chuyên ngành:

1. Đối với kỹ sư chính quy:
 - (a) Từ K61 trở về trước: Ngành Kỹ thuật phần mềm
 - (b) Từ K62 trở về sau: Ngành Khoa học máy tính
2. Đối với cử nhân:
 - (a) Ngành Công nghệ thông tin
3. Đối với chương trình EliteTech:
 - (a) Chương trình Việt Nhật/KSTN: Ngành Công nghệ thông tin
 - (b) Chương trình ICT Global: Ngành Information Technology
 - (c) Chương trình DS&AI: Ngành Khoa học dữ liệu

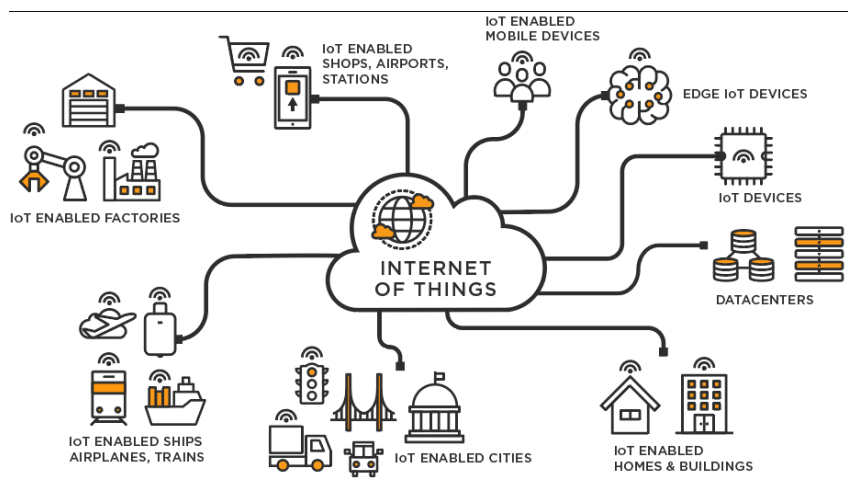
A.2 Đánh dấu (bullet) và đánh số (numering)

Việc sử dụng danh sách trong LaTeX khá đơn giản và không yêu cầu sinh viên phải thêm bất kỳ gói bổ sung nào. LaTeX cung cấp hai môi trường liệt kê đó là:

- Đánh dấu (bullet) là kiểu liệt kê không có thứ tự. Để sử dụng kiểu liệt kê đánh dấu, chúng ta khai báo như sau

```
\begin{itemize}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{itemize}
```
- Đánh số (numering) là kiểu liệt kê có thứ tự. Để sử dụng kiểu liệt kê đánh số, chúng ta khai báo như sau

```
\begin{enumerate}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
```



Hình A.1: Internet vạn vật

```
\item ...
\end{enumerate}
```

Chú ý các nội dung trình bày trong cả hai môi trường liệt kê theo sau lệnh `\item`. Ngoài ra LaTeX còn cung cấp một số kiểu liệt kê khác, sinh viên có thể tham khảo tại <https://www.overleaf.com/learn/latex/Lists>

A.3 Cách thêm bảng

Col1	Col2	Col2	Col3
1	6	87837	787
2	7	78	5415
3	545	778	7507
4	545	18744	7560
5	88	788	6344

Bảng A.1: Table to test captions and labels.

Bảng A.1 là ví dụ về cách tạo bảng. Tất cả các bảng biểu phải được đề cập đến trong phần nội dung và phải được phân tích và bình luận. Chú ý: Tạo bảng trong LaTeX khá phức tạp và mất thời gian, vì vậy sinh viên có thể sử dụng các công cụ hỗ trợ tạo bảng (Ví dụ: <https://www.tablesgenerator.com/>). Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong LaTeX tại link <https://www.overleaf.com/learn/latex/Tables>.

A.4 Chèn hình ảnh

Hình A.1 là ví dụ về cách chèn ảnh. Lưu ý chú thích của hình vẽ được đặt ngay dưới hình vẽ. Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong LaTeX tại https://www.overleaf.com/learn/latex/Inserting_Images.

Chú ý, tất cả các hình vẽ phải được đề cập đến trong phần nội dung và phải được phân tích và bình luận.

A.5 Tài liệu tham khảo

Cách liệt kê

Áp dụng cách liệt kê theo quy định của IEEE. Ví dụ của việc trích dẫn như sau **peterson2007computer**. Cụ thể, sinh viên sử dụng lệnh `\cite{}` như sau **NguyenThucHai**. Chỉ những tài liệu được trích dẫn thì mới xuất hiện trong phần Tài liệu tham khảo. Tài liệu tham khảo cần có nguồn gốc rõ ràng và phải từ nguồn đáng tin cậy. Hạn chế trích dẫn tài liệu tham khảo từ các website, từ wikipedia.

Các loại tài liệu tham khảo

Các nguồn tài liệu tham khảo chính là sách, bài báo trong các tạp chí, bài báo trong các hội nghị khoa học và các tài liệu tham khảo khác trên internet.

A.6 Cách viết phương trình và công thức toán học

Các gói amsmath, amssymb, amsfonts hỗ trợ viết phương trình/công thức toán học đã được bổ sung sẵn ở phần đầu của file main.tex. Một ví dụ về tạo phương trình (A.1) như sau

$$F(x) = \int_b^a \frac{1}{3}x^3 \quad (\text{A.1})$$

Phương trình A.1 là ví dụ về phương trình tích phân. Một phương trình khác không được đánh số thứ tự (gán nhãn)

$$x[t_n] = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X[f_k] e^{j2\pi nk/N}$$

Phương trình này thể hiện phép biến đổi Fourier rời rạc ngược (IDFT).

A.7 Qui cách đóng quyển

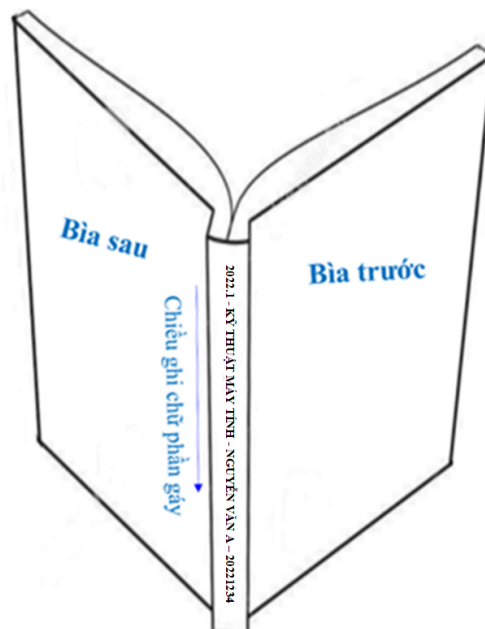
Phần bìa trước chế bản theo qui định; bìa trước và bìa sau là giấy liền khổ. Sử dụng keo nhiệt để dán gáy khi đóng quyển thay vì sử dụng băng dính và dập ghim như mô tả ở Hình A.3 Phần gáy ĐATN cần ghi các thông tin tóm tắt sau: Kỳ làm ĐATN - Ngành đào tạo - Họ và tên sinh viên - Mã số sinh viên. Ví dụ:

2022.1 - KỸ THUẬT MÁY TÍNH - NGUYỄN VĂN A - 20221234

Qui cách ghi chữ phần gáy như hình dưới đây:



Hình A.2: Qui cách đóng quyển đồ án



Hình A.3: Qui cách đóng quyển đồ án

B. ĐẶC TẢ USE CASE

Nếu trong nội dung chính không đủ không gian cho các use case khác (ngoài các use case nghiệp vụ chính) thì đặc tả thêm cho các use case đó ở đây.

B.1 Đặc tả use case “Tổng kê tình hình mượn sách”

...

B.2 Đặc tả use case “Đăng ký làm thẻ mượn”

...